

Cahier des charges : Assistant robotique retail

—
Latence perçue sub-seconde / architecture agent / interface et respect RGPD

Projet d'études

Périmètre V1 : Rayon capillaire (shampoings et soins capillaires)

Plateforme : Robot humanoïde Pepper + Mac Apple Silicon M4 Pro

Contexte : Démonstrateur fonctionnel avec validation par scénarios

—

Résumé du projet

Ce projet vise à concevoir et valider un assistant robotique pour le secteur parapharmacie, capable d'assister les clients directement en rayon avec une interaction naturelle. Le démonstrateur repose sur le robot humanoïde Pepper comme interface « d'expression » (voix, gestes, LEDs, tablette tactile).

Nos premières explorations nous conduisent à choisir pour ce démonstrateur un modèle de langage externe pour la compréhension et la génération conversationnelle. Ce type de modèle est communément dénommé *speech-to-speech*.

Trois défis structurants guident la conception :

1. **Il faut tendre vers une latence perçue inférieure à 1,2 seconde** : Cette contrainte critique nous impose par construction une architecture hybride local/cloud optimisée. Nous en développerons les raisons et les composantes dans ce document.
2. **La mise en production d'une interface naturelle pilotée par un LLM externe** : Cette architecture nous impose de nous orienter vers une solution agent/interface découplée. L'enjeu est de nourrir le LLM en temps réel avec des données dites « produit ».
3. **Approche RGPD by design** : sans reconnaissance faciale, sans stockage persistant, et avec une minimisation des données en local pour la continuité de session sur une base strictement proportionnée. En effet, la continuité de session anonyme sans reconnaissance faciale semble au premier abord insurmontable. Il existe néanmoins des solutions d'architecture réalistes et strictement proportionnée que nous présenterons.

Pour répondre à ces défis, l'architecture que nous proposons est de combiner une vision locale optimisée (VLM sur MLX Apple Silicon, latence 200-350ms) et d'utiliser une conversation cloud en streaming (*speech-to-speech* OpenAI Realtime, première syllabe <300ms). À cette approche globale il faut associer des techniques dites de « masquage perceptif ». Il s'agit de fait de tromper l'attente de l'utilisateur avec des micro-phrases locales et des animations synchronisées.

La validation reposera sur 20 scénarios dit cœurs testés en interne. Ce jeu de scénarios nous permet d'envisager les situations les plus courantes. De fait la satisfaction utilisateur n'est malheureusement pas mesurable au travers d'un jeu d'essai fait en interne mais nous restons dans un cadre universitaire sans pouvoir réaliser de vrais tests en magasin et avec une accessibilité au robot Pepper minimale. Le périmètre V1 se limitera au rayon capillaire (30 produits) et exclura explicitement les médicaments et conseils médicaux. Il convient de noter que ces 20 scénarios comprennent 4 scénarios de sécurité indéniables.

De manière synthétique pour la partie VLM, on part sur un système qui identifie les produits par analyse visuelle du packaging (stratégie VLM et en parallèle une tentative de lectures de code barre

si celui-ci apparaît) avec un repli automatique sur une lecture de code-barres EAN-13 plus stricte en cas d'incertitude en demandant à l'utilisateur de présenter ce code barre à la caméra du robot.

1. Introduction et contexte

1.1 Contexte général

Après une courte étude, il apparaît que le secteur de la parapharmacie se caractérise par une densité élevée de références commerciales, en particulier dans certains rayons comme le capillaire où l'on peut dénombrer entre 100 et 300 références pour une surface de vente moyenne. Cette profusion de produits aux packagings similaires et aux promesses marketing proches génère de multiples confusions pour les clients, qui hésitent souvent entre plusieurs références sans disposer du temps ou de l'expertise pour comparer les compositions, usages recommandés et précautions d'emploi.

Parallèlement, le personnel en parapharmacie fait face à une charge de travail croissante, notamment aux heures d'affluence, limitant sa disponibilité pour accompagner chaque client dans le choix de produits non soumis à prescription, pour autant étonnement les marges sur ces produits constituent une part importante des revenus des officines. Le démonstrateur semble donc être potentiellement être un outil opportun dans ce contexte.

1.2 Robots humanoïdes en retail : enseignements

Les robots humanoïdes, et en particulier le Pepper de SoftBank Robotics, ont fait l'objet de plusieurs déploiements expérimentaux en contexte retail (Japan Airlines pour l'accueil passagers dans ses boutiques de merchandising, Carrefour pour l'orientation clients, diverses enseignes pour la promotion produits).

Ces expérimentations ont mis en évidence plusieurs enseignements critiques :

- L'expressivité robotique (gestes, orientation corporelle, signaux lumineux) favorise l'engagement des utilisateurs comparativement aux bornes interactives statiques, on sort ainsi clairement de l'interface écran/machine.
- Les guidelines B2BD (*Business to Business Development*) de SoftBank insistent sur la nécessité de maintenir Pepper "vivant" pendant l'interaction : animations continues, variations d'attention, postures expressives.
- La latence d'interaction constitue un frein majeur à l'acceptabilité : les systèmes présentant des temps de réponse supérieurs à 2-3 secondes génèrent frustration et abandon. Les retours terrain rapportent systématiquement ce problème comme critique.
- L'intégration de capacités conversationnelles avancées reste limitée par les contraintes de traitement embarqué sur les robots actuels. Les architectures tout-cloud génèrent des latences cumulées de 2 à 5 secondes, jugées inacceptables.

1.3 Positionnement du robot Pepper

Pepper est un robot humanoïde conçu spécifiquement pour l'interaction sociale. Ses caractéristiques principales incluent :

- Des capteurs de perception (capteur 3D pour la détection de présence, caméras vidéo pour l'analyse visuelle, 4 microphones directionnels pour la capture audio).
- Des actionneurs expressifs (17 moteurs permettant l'orientation corporelle et les gestes, LEDs programmables pour les signaux visuels, 2 haut-parleurs pour la restitution vocale).
- Une interface tablette tactile 10.1" pour l'affichage d'informations.

Le système NAOqi propriétaire (OpenNAO basé sur Linux Gentoo) est pilotable via SDK Python. Les recommandations B2BD déconseillent notamment l'utilisation de LEDs rouges (perçues comme signal de colère) et recommandent des mouvements crédibles anatomiquement.

Le présent projet s'appuie autant que possible sur ces recommandations pour concevoir une interaction expressive et engageante. L'objet est de garder le plus possible le "moteur expressif" de Pepper : animations et posture doivent rester locales au robot plutôt que d'envoyer des micro-angles moteur à la main qui serait de plus un chantier trop exigeant.

2. Problématique et défis techniques

2.1 Énoncé de la problématique

Comment concevoir un assistant robotique en environnement retail qui combine :

1. Une latence moyenne perçue inférieure à 1,2 seconde (contrainte d'utilisabilité critique pour l'acceptabilité).
2. Une interface naturelle exploitant un modèle de langage externe au travers d'une architecture agent/interface.
3. Une approche RGPD By design sans recours à la reconnaissance faciale ou à la conservation de données biométriques (contrainte réglementaire et éthique).

Cette problématique articule trois dimensions complémentaires : dimension technique (optimisation latence), dimension architecturale (découplage agent/interface), et dimension réglementaire (conformité RGPD by design).

2.2 Défi 1 : Latence perçue sous 1,2 seconde

2.2.1 Analyse du problème

Latence incompressible côté robot : le flux micro Pepper est fourni en trames bufferisées par la couche audio (ordre de grandeur ~170 ms annoncé dans la documentation Pepper en 4 canaux à 48 kHz, à confirmer par mesure), ce qui constitue un plancher de latence avant toute Voice Activity Détection ou envoi réseau.

Une interaction utilisateur complète implique plusieurs étapes de traitement : détection fin de parole utilisateur (VAD, *Voice Activity Detection* 100-200ms), analyse visuelle du produit présenté (VLM, 200-3000ms selon architecture), génération de réponse conversationnelle (LLM, 300-1500ms), synthèse vocale (TTS, 200-500ms), et latence réseau si cloud (50-200ms aller-retour).

Dans une architecture naïve tout-cloud, ces latences s'accumulent pour atteindre 850-5400ms, soit 1 à 5 secondes perçues par l'utilisateur. La littérature en interaction homme-machine établit qu'au-delà de 1 seconde et précisément dès 1,2s l'utilisateur perçoit l'attente comme frustrante et perd le sentiment de fluidité de l'interaction.

Précision critique sur la latence perçue :

Il est essentiel de distinguer la latence perçue comme problématique de la durée totale de la réponse. Dans une conversation humaine naturelle, un silence supérieur à 1,2 seconde génère un malaise conversationnel : l'utilisateur se demande si le système a compris, s'il traite la demande, ou si la communication est rompue. En revanche, une fois que le système commence à répondre, même si la phrase complète prend 3 à 4 secondes, l'attente psychologique disparaît : l'échange est engagé, l'utilisateur écoute la réponse qui se construit.

Cette observation psychologique structure l'ensemble de notre architecture : l'objectif n'est pas de réduire la durée totale de chaque réponse, mais d'éliminer le silence générateur d'un vide ressenti comme un blocage. Ainsi, une réponse dont la première syllabe arrive en 850ms mais dont la phrase complète nécessite 3,5 secondes sera perçue comme fluide et réactive. À l'inverse, une réponse très courte (1 seconde au total) précédée de 2 secondes de silence sera perçue comme lente et frustrante, même si la durée totale est inférieure.

Le streaming audio (OpenAI Realtime *speech-to-speech*) devient donc une brique structurante : il permet de générer la première syllabe en 200-400ms après détection de fin de parole, puis de poursuivre la génération pendant que Pepper restitue déjà l'audio. L'utilisateur n'attend jamais : dès 850ms, il écoute.

Référentiel de mesure (pour éviter toute confusion)

- **T1** : instant où le système détecte la fin de parole (événement VAD / "speech_stopped" reçu par le Mac).
- **T2** : première réaction perceptible locale (LED/animation/micro-phrase).
- **T3** : début effectif de la réponse audio (premier buffer envoyé à sendRemoteBufferToOutput, donc début du playback Pepper).

On mesure T2-T1 (réaction) et T3-T1 (démarrage audio). Les "planchers" (buffering micro, silence VAD, chunking) influencent T1, ils ne sont pas des termes qu'on additionne après.

2.2.2 Solution proposée : architecture hybride optimisée

Nous combinons plusieurs approches pour réduire la latence perçue :

1/ Optimisation d'un VLM local via MLX (Apple Silicon)

Le choix de MLX est structurant pour le projet. Il permet d'atteindre des latences VLM de 200-350ms sur Mac M4 Pro (modèle Qwen2-VL-7B quantisé 4-bit), contre 2000-3000ms avec PyTorch CPU sur la même machine. Cette optimisation exploite le GPU Metal, le Neural Engine et la mémoire unifiée des puces Apple Silicon. Sans cette optimisation MLX, le projet ne serait pas viable avec l'objectif de latence fixé. Les frameworks généralistes comme PyTorch sur CPU génèrent des latences de 2-3s pour l'inférence VLM, qui cumulées avec la latence conversationnelle conduiraient à des temps de réponse totaux de 3-5s, bien au-delà du seuil d'acceptabilité. Il existe évidemment des solutions alternatives avec un PC et des cartes nvidia dont nous ne disposons pas.

La quantization 4-bit supportée nativement par MLX permet de réduire l'empreinte mémoire (14GB → 5GB pour Qwen2-VL-7B) tout en maintenant une qualité d'inférence acceptable. Cette approche devra être validée par des tests réels lors de la réalisation du projet.

2/ Streaming audio speech-to-speech

L'utilisation de l'API OpenAI Realtime en mode speech-to-speech permet de générer la première syllabe de réponse en 200-400ms après fin de parole utilisateur, avec un streaming continu de l'audio. Contrairement à une architecture pipeline classique (Speech-To-Text + LLM + Text-to-Speech séquentiel), le streaming permet à Pepper de commencer à parler dès que les premiers tokens audio sont générés, sans attendre la phrase complète.

Masquage perceptif de la latence

Tant qu'une réaction visible survient dans les 300ms suivant la fin de parole utilisateur, l'utilisateur perçoit le système comme réactif même si la réponse complète prend 800-1000ms. Nous déclencherons immédiatement (<100ms) une micro-phrase locale avec une petite banque locale de

phrases pré-enregistrées comme "D'accord", "Je regarde", ou "Je vais vous répondre", couplée à une animation (hochement de tête) et un changement de LEDs (bleu écoute → vert analyse).

Cache scène et pré-loading

Si le même produit reste visible dans le cadre caméra (détecté par analyse de similarité visuelle de l'image), le système ne re-scane pas le produit lors des questions suivantes. Cette optimisation économise environ 300ms par question additionnelle sur le même produit.

2.2.3 Budget latence cible

L'objectif est d'atteindre une réaction visible en moins de 300ms (micro-phrase + LED + animation) et un début de réponse audio complète en moins de 1000ms. Nous prenons la précaution d'insister sur le mot « objectif ».

Important : les valeurs ci-dessous ne s'additionnent pas.

Elles décrivent deux métriques différentes : (i) la réaction perçue (casser le silence) et (ii) le début effectif de la réponse audio. Les éléments comme le buffering Pepper et le réglage VAD définissent quand la fin de parole est détectée, mais ne s'ajoutent pas "après coup" aux budgets de réaction.

La décomposition typique est :

La réaction perçue (casser le silence) :

- VAD - Voice Activity Detection = fin de parole : 150ms
- Réaction immédiate visible : 100ms (total 250ms ✓ < 1s)
- VLM identification si nécessaire : 300ms
- LLM génération première syllabe : 300ms
- Début audio réponse : ~850ms total

Le début effectif de la réponse audio :

- Buffering micro Pepper (framing) : ~170 ms (Grandeur annoncé par la documentation Pepper)
- Détection fin de parole (VAD, incl. silence) : 150–500 ms selon réglage
- Réaction visible locale (LED + animation + backchannel) : <100 ms

Le budget "première syllabe" dépend fortement du réglage `silence_duration_ms` (fin de parole), donc la métrique clé est "réaction visible" plus "début audio après événement `speech_stopped`"

2.3 Défi 2 : Architecture agent/interface découplée

2.3.1 Principe de séparation

Nous proposons une séparation stricte entre :

- **Interface (Pepper)** : expressivité robotique (gestes, posture, LEDs, voix synthétisée, tablette tactile).
- **Agent (LLM externe)** : intelligence conversationnelle (intent understanding dans la documentation OpenAI, gestion du contexte, génération des réponses).
- **Traitement local (Mac)** : vision (VLM), base produits (SQLite), orchestration d'une cinétique de machine à états assez simple.

Nota : Une machine à états, c'est un modèle logique qui décrit un système comme un ensemble d'états, des transitions entre ces états, déclenchées par des événements.

Cette architecture présente plusieurs avantages. Le premier est une architecture de briques fonctionnelles indépendantes qui peuvent s'améliorer sans modification du code Pepper, et d'upgrader la vision (Qwen2-VL → modèle plus performant) sans modification de la conversation. La réduction de la complexité embarquée signifie que Pepper porte l'expressivité et la capture sensorielle, sans traitement lourd, on tirera donc le meilleur du robot. Enfin, la testabilité et le debug sont améliorés : l'agent est testable indépendamment (simulation conversations sans robot), l'interface testable indépendamment (animations, gestes sans LLM).

2.3.2 Flux d'interaction

L'utilisateur parle → Pepper capture audio (4 micros) vers Mac → Mac stream audio vers Agent cloud (OpenAI Realtime) → Agent génère réponse (streaming) → Mac reçoit audio stream vers Pepper speakers avec des consignes d'animation.

En parallèle, le Mac orchestre déclenche les animations Pepper (gestes, LEDs) synchronisées avec la parole en suivant les consignes si nécessaire. L'orchestration locale (Mac) agit comme chef d'orchestre coordonnant interface et agent, sans que ces deux composants communiquent directement.

2.4 Défi 3 : RGPD by design sans reconnaissance faciale

2.4.1 Contraintes réglementaires

Le RGPD impose des contraintes strictes sur les données biométriques (Article 9 - données sensibles). Tout traitement permettant l'identification unique d'une personne physique constitue un traitement biométrique nécessitant :

- Une base légale renforcée (consentement par action positive ou intérêt légitime démontré)
- Une information claire et préalable des personnes concernées
- Des mesures de sécurité techniques et organisationnelles appropriées
- Le respect des principes de minimisation et limitation de la durée de conservation

Enjeux spécifiques au contexte retail :

En environnement retail grand public, deux problématiques se posent :

1. Friction utilisateur : Obtenir un consentement formel via formulaire pour chaque interaction serait contre-productif et créerait une barrière majeure à l'usage.

2. Proportionnalité du traitement : Les données traitées doivent être strictement nécessaires à la finalité poursuivie (continuité conversationnelle) et limitées dans le temps.

Notre réponse : transparence et consentement par action positive

Nous mettons en place un double mécanisme d'information transparent :

1. Information vocale (phrase d'accueil du robot)

Dès le premier contact, Pepper prononce :

"Bonjour ! Je suis Pepper, un robot démonstrateur. Pour fluidifier notre échange, j'analyse temporairement votre présence. Toutes les données sont effacées après notre conversation. En continuant, vous acceptez ces conditions. Comment puis-je vous aider ?"

Cette phrase remplit trois fonctions :

- Information claire sur le traitement temporaire
- Notification de la durée limitée (effacement post-conversation)
- Obtention du consentement par action positive (continuer l'interaction)

2. Affichage permanent sur tablette

Un message visible lors de la phase initiale d'accueil sur la tablette de Pepper détaille les garanties :

ROBOT DÉMONSTRATEUR

Ce robot utilise temporairement :

- Analyse de présence (silhouette)
- Caractéristiques vocales générales
- Traitement conversationnel cloud (audio dialogue uniquement)
- ✓ Aucune reconnaissance faciale
- ✓ Aucune identification personnelle
- ✓ Données effacées après 7 minutes max
- ✓ Aucune conservation, aucun partage

En engageant la conversation, vous acceptez ce traitement temporaire.

Base légale :

- **Article 6.1.f RGPD** : Intérêt légitime (amélioration de l'expérience utilisateur par continuité conversationnelle)
- **Article 9.2.a RGPD** : Consentement explicite pour le traitement de données biométriques (obtenu par information préalable + action positive)

Cette approche permet de concilier fluidité d'usage et conformité réglementaire dans un contexte d'un démonstrateur académique.

2.4.2 Solution : continuité anonyme

Cas d'usage : continuité de session

Scénario illustratif :

Une cliente s'approche de Pepper et lui demande : « Avez-vous un shampoing pour les cheveux blonds ? »

Pepper répond : « Oui. Je vous recommande le shampoing Klorane pour cheveux clairs. Il se trouve sur la droite du rayon, en haut à gauche. »

Pendant ce temps, une autre cliente interpelle Pepper pour une question différente.

Quatre minutes plus tard, la première cliente revient avec le shampoing Klorane à la main et demande : « Dis-moi, est-ce que ce shampoing est bio ? »

Sans continuité de session, Pepper ne ferait pas le lien et traiterait la question comme nouvelle, créant confusion et frustration.

Avec notre approche, Pepper reconnaît la continuité (sans identifier la personne) et répond : « Je vois que vous avez pris le shampoing Klorane que je vous avais recommandé. Oui, il est bio, et je peux également vous fournir la liste complète de ses composants. »

Solution technique : double signature temporaire non-identifiante

Architecture Privacy by Design

Notre approche repose sur un mécanisme de continuité de session sans identification utilisant une représentation approximative et non réversible. Le système combine deux vecteurs de signature complémentaires :

1. Vecteur silhouette (features visuelles bas niveau)

- Position 3D approximative (capteur profondeur Pepper)
- Taille relative (bounding box normalisée)
- Couleurs dominantes vêtements (histogramme HSV simplifié, 8 bins)
- **Génération** : Vecteur mathématique de dimension réduite (≈ 64 dimensions)
- **Caractéristiques** : Non réversible en image, pas d'analyse de traits faciaux, pas de reconnaissance biométrique au sens strict

2. Vecteur vocal (caractéristiques acoustiques bas niveau)

- Fréquence fondamentale (F0) moyenne
- Formants principaux (F1, F2, F3)
- Énergie spectrale dans 3-4 bandes de fréquences
- **Extraction** : Échantillon audio de 2-3 secondes lors de la première prise de parole
- **Génération** : Vecteur mathématique de dimension réduite (≈ 32 dimensions)
- **Caractéristiques** : Non réversible en audio, pas d'empreinte biométrique unique (caractéristiques acoustiques générales)

Justification du double vecteur :

- Améliore significativement la fiabilité (réduction des faux positifs/négatifs)
- Compense les variations individuelles (changement de position, variation vocale)
- Maintient la conformité RGPD : signatures mathématiques temporaires, non réversibles, non transmises

Traitement et stockage

Localisation exclusive : RAM locale Pepper/Macbook Pro M4pro

- Les deux signatures (silhouette + voix) : 96 dimensions totales, ≈ 384 octets
- Métadonnées session : timestamp dernier contact, contexte conversationnel (≈ 2 Ko)
- **Aucune transmission réseau** des signatures (seul l'audio conversationnel transite vers l'API OpenAI Realtime pour le speech-to-speech)
- **Aucune persistance disque** : volatilité garantie par architecture RAM-only

Time To Live (TTL) : 7 minutes maximum

- Paramètre ajustable selon contexte (valeur par défaut : 7 minutes)
- Auto-expiration automatique sans intervention
- Réinitialisation complète à l'expiration

Mécanisme de reconnaissance de continuité :

1. Détection présence utilisateur (capteurs Pepper)
2. Calcul signatures silhouette + voix
3. Comparaison par distance cosinus avec signatures en RAM
4. Si distance < seuil (0.85) ET TTL non expiré $\rightarrow$ continuité détectée
5. Sinon $\rightarrow$ nouvelle session

Comportement interface :

- **Continuité détectée** : Pepper affiche directement la fiche produit précédente, dit "Puis-je vous aider ?" sans ré-accueil complet
- **Nouvelle session** : Accueil standard avec phrase d'information RGPD
- **Bouton tablette** : "Nouvelle conversation" permet de forcer une réinitialisation manuelle

Traitement vocal externalisé (OpenAI Realtime API)

Séparation claire des traitements :

- **Local (Pepper)** : Calcul des signatures silhouette + voix → jamais transmises
- **Cloud (OpenAI)** : Speech-to-speech conversationnel uniquement (audio brut in/out)

Garanties contractuelles OpenAI :

- Data Processing Agreement (DPA) RGPD Business Terms
- Pas d'utilisation des données API pour entraînement de modèles
- Pas de conservation post-session (mode streaming)
- Conformité transferts hors UE (clauses contractuelles types)

Référence documentation : <https://openai.com/policies/business-terms>

2.4.3 Conformité RGPD : principes et garanties

Cette section détaille l'application concrète des principes RGPD à notre architecture, l'analyse des risques résiduels, et une comparaison avec les systèmes commerciaux standards.

Principes RGPD appliqués

Principe RGPD	Application concrète
Licéité	Base légale : intérêt légitime (Art. 6.1.f) + consentement biométrie (Art. 9.2.a)
Minimisation	Uniquement vecteurs bas niveau (96 dimensions), pas de photos/audio conservés, dimensions réduites
Limitation finalité	Continuité conversationnelle uniquement, pas de profilage, pas d'autres usages
Limitation conservation	7 minutes maximum, RAM volatile, purge automatique
Exactitude	Signatures approximatives (tolèrent variations naturelles)
Intégrité / confidentialité	Traitement local sécurisé, pas de réseau pour signatures, DPA cloud
Transparence	Information vocale + tablette avant toute interaction
Privacy by Design	Architecture conçue dès l'origine sans reconnaissance faciale stricte

Analyse de risques et proportionnalité

Risque	Probabilité	Impact	Mesures d'atténuation
Confusion entre utilisateurs (faux positif)	Faible	Faible	Double vecteur, seuil conservateur (0.85)
Non-détection continuité (faux négatif)	Moyenne	Très faible	Acceptable (retour accueil standard)

Interception données réseau	Très faible	Faible	TLS 1.3, pas de signatures transmises
Persistance involontaire	Très faible	Faible	Architecture RAM-only, pas d'accès disque

Conclusion de l'analyse : Risques résiduels négligeables pour un démonstrateur temporaire sans conséquence pour l'utilisateur.

Comparaison avec systèmes commerciaux

Critère	Notre système	Assistants vocaux (Alexa, Google)	Caméras retail standard
Conservation	7 min max	Indéfinie (avec compte)	30+ jours
Identification unique	Non	Oui (compte)	Oui (vidéo)
Réversibilité	Non (vecteurs)	Partielle	Oui (vidéo complète)
Base de données	Non	Oui	Oui

Conclusion : Notre système est significativement plus protecteur que les standards industriels actuels en matière de respect de la vie privée et de conformité RGPD.

2.4.4 Cadre de test et développement

Pour permettre le développement itératif et l'amélioration du système tout en maintenant la conformité RGPD, nous définissons un cadre de test structuré en deux modes distincts.

Positionnement éthique et technique

Traitement vocal assumé :

Nous assumons un traitement vocal minimal strictement nécessaire à la continuité de session :

- Extraction de caractéristiques acoustiques bas niveau (F0, formants, énergie)
- Comparaison locale par similarité vectorielle (distance cosinus)
- **Aucun audio brut conservé** au-delà du traitement immédiat
- **Aucune empreinte biométrique vocale unique** (caractéristiques générales, pas de signature individuelle forensique)
- Signatures temporaires (RAM + TTL) servant uniquement à éviter la confusion entre utilisateurs à quelques minutes d'intervalle

Ce traitement est proportionné car :

1. **Finalité légitime** : amélioration significative de l'expérience utilisateur
2. **Nécessité** : sans continuité, l'interaction devient incohérente et frustrante
3. **Minimisation** : données strictement nécessaires, durée minimale
4. **Absence d'alternative** : pas de solution moins intrusive offrant le même service

Axiomes du cadre de développement

Axiome 1 : Mode production (PRODUCTION_MODE = true)

Un paramètre de configuration active le mode production, qui bloque toute forme d'enregistrement non essentiel :

Axiome 2 : Mode développement avec consentement explicite

Restrictions en mode production :

- Pas de logs détaillés (uniquement erreurs critiques anonymisées)
- Pas de traces session persistantes
- Pas de statistiques nominatives
- Pas de télémétrie développeur

Objectif : Zéro donnée conservée au-delà de la session RAM (7 min max)

Les membres de l'équipe projet donnent leur consentement éclairé et documenté pour :

Autorisations en mode développement :

- Enregistrement de sessions de test (audio + logs détaillés)
- Analyse des performances du système de continuité
- Conservation temporaire (≤ 30 jours) à des fins de débogage

Documentation du consentement :

- Formulaire signé par chaque participant test
- Durée de conservation explicite (30 jours max)
- Droit de retrait à tout moment
- Destruction garantie post-test

Procédure de transition vers la production

Avant toute démonstration publique ou test avec utilisateurs externes universitaires, les étapes suivantes sont obligatoires :

Étape	Action requise
1	Activation du mode production : PRODUCTION_MODE = true dans la configuration
2	Suppression des données de développement : Purge complète des logs et enregistrements de test
3	Vérification technique : Audit de l'absence de logs/traces persistants
4	Validation affichage : Vérification de la présence du message tablette et de la phrase d'accueil

Synthèse

Ce cadre dual (développement/production) permet de :

- ✓ **Développer et améliorer** le système de façon itérative avec consentement explicite des testeurs
- ✓ **Garantir la conformité RGPD** en production avec zéro conservation de données
- ✓ **Tracer la transition** entre les deux modes de façon documentée et vérifiable
- ✓ **Maintenir la transparence** vis-à-vis des utilisateurs finaux et de l'équipe pédagogique

Cette approche démontre qu'un développement rigoureux et une conformité réglementaire stricte sont conciliables dans un contexte académique.

2.5 Contributions attendues

Les solutions architecturales décrites dans les sections précédentes visent trois contributions au domaine de l'interaction robotique en environnement retail.

Contribution 1 :

Démonstration qu'une architecture hybride local/cloud exploitant MLX pour la vision et le streaming audio pour la conversation permet d'atteindre une latence perçue compatible avec l'interaction fluide (objectif < 1,2 seconde). Le projet mesurera empiriquement les latences obtenues et l'écart entre temps de traitement technique et temps perçu par l'utilisateur grâce aux techniques de masquage perceptif.

Contribution 2 :

Validation d'une architecture agent/interface découplée via un orchestrateur central gérant la coordination entre le robot Pepper, les traitements locaux et les services cloud. Cette approche facilite l'évolution indépendante des composants et simplifie le développement itératif. Le code de l'orchestrateur sera documenté comme référence d'implémentation pour d'éventuels travaux futurs.

Contribution 3 :

Preuve de concept d'un mécanisme de continuité de session anonyme conforme au RGPD, basé sur des signatures perceptuelles non-identifiantes (silhouette + voix) stockées en RAM volatile. Le projet évaluera si cette approche fournit une continuité conversationnelle suffisante pour améliorer l'expérience utilisateur tout en respectant les principes de Privacy by Design, et documentera ses limites par rapport aux techniques biométriques classiques.

Ces contributions seront validées via les 20 scénarios de test décrits en section 6, dans le cadre des limites assumées d'un démonstrateur académique (section 7).

3. Démarche et travail préparatoire

3.1 Analyse des déploiements existants

Notre première recherche s'est orientée naturellement vers les déploiements existants de robots humanoïdes en retail. Pepper a été testé chez Carrefour pour orienter les clients, chez Japan Airlines dans des boutiques, dans diverses enseignes pour promouvoir des produits.

Les retours terrain révèlent systématiquement les mêmes obstacles. Les temps de réponse dépassent deux ou trois secondes et les utilisateurs abandonnent. Quand les réponses arrivent rapidement, elles sonnent scriptées et perdent rapidement leur intérêt. Ces deux constats ont orienté l'ensemble du projet. Réduire le temps de réponse et rendre la conversation naturelle sont devenus nos priorités.

3.2 Exploration technique et choix d'outils

3.2.1 Recherche des architectures conversationnelles et projets académiques

Une revue a été menée sur des dépôts publics et documents accessibles portant sur Pepper, NAOqi, streaming audio, reconnaissance vocale, synthèse vocale, et gestion de l'écho. L'objectif n'était pas de trouver un système à réutiliser tel quel, mais de repérer des traitements et des choix d'architecture déjà éprouvés.

Aucun dépôt public clairement identifié ne décrit de bout en bout une solution Pepper + OpenAI Realtime en speech-to-speech temps réel, avec gestion documentée des cas difficiles comme le recouvrement audio, le bruit magasin, ou l'interruption. Ce constat indique surtout une absence de référence publiée directement comparable, pas l'impossibilité technique.

Plusieurs projets académiques ont fourni des enseignements concrets :

- **TU Wien (Vienna University of Technology) - "pepperspeechrecognition" :**

Le projet de Johannes Bramauer propose un module NAOqi en Python qui utilise Google Speech Recognition pour Pepper, avec des mécanismes concrets autour de la détection de début et fin de parole et de l'adaptation au bruit. Même lorsqu'on vise un *speech-to-speech*, la fiabilité dépend d'abord d'une bonne gestion de la parole en entrée : détection, calibration, robustesse au bruit. Ce travail fournit une base d'inspiration pour l'outillage de tests et la structuration d'un module audio côté Pepper, notamment l'abonnement micro, le traitement en blocs, et les événements de parole.

- **Université d'Osnabrück - "pepper_speech_recognition" :**

Ce dépôt souligne des limites pratiques du micro-array interne de Pepper dans certains scénarios, au point qu'un micro externe type Respeaker est mentionné comme alternative. Il insiste aussi sur des contraintes terrain comme la stabilité réseau. Le dépôt documente une stratégie très directe contre l'auto-écoute : quand Pepper parle, le flux audio envoyé au service de reconnaissance est remplacé par des zéros, afin d'éviter que la voix du robot ne soit interprétée comme celle du client. Cette stratégie illustre un choix d'architecture assumé qui privilégie la stabilité et la simplicité au détriment du *barge-in* (interruption vocale). Elle montre aussi que dans un environnement réel, les questions réseau et les boucles audio deviennent vite centrales.

- **Umeå University + University of Hamburg - WoZ4U :**

WoZ4U, développé par l'Intelligent Robotics Group en collaboration avec le Knowledge Technology Group, propose une interface de type Wizard-of-Oz pour piloter Pepper, incluant du monitoring et de l'enregistrement en temps réel des microphones et caméras. L'outil rappelle un point méthodologique essentiel : pour progresser rapidement, il faut rendre les essais reproductibles, c'est-à-dire capturer, archiver, rejouer. Ceci devient particulièrement important quand l'accès au robot est intermittent, avec un robot disponible au laboratoire seulement sur rendez-vous et un développement sur Mac le reste du temps.

- **PepperChat (hsaine) : intégration non académique :**

Ce projet illustre une chaîne de dialogue Pepper + IA avec une étape de STT cloud (Watson) et un échange de type LLM. Même si le service et la pile technique diffèrent, il met en évidence les problèmes récurrents d'intégration : enchaînement capture audio, interprétation, réponse, et les points d'attention pratiques pour un dialogue temps réel.

L'API OpenAI Realtime a nécessité une lecture approfondie de la documentation au-delà des exemples de base. Les mécanismes de streaming audio, les réglages de détection de fin de parole via *Voice Activity Detection* sémantique, la gestion du contexte sur plusieurs échanges, et les possibilités d'interruption de réponse (*interrupt_response*) demandent du temps pour être maîtrisés. Cette API *speech-to-speech* génère la première syllabe en moins de 300ms après détection de fin de parole, un gain décisif comparé aux systèmes classiques qui enchaînent transcription, génération de texte, puis synthèse vocale de manière séquentielle.

Pour la vision, plusieurs frameworks ont été testés. PyTorch sur CPU nécessite 2 à 3 secondes pour analyser une image avec un modèle comme Qwen2-VL-7B. MLX sur Apple Silicon ramène ce temps à 200-350ms en exploitant le GPU Metal et la mémoire unifiée du M4 Pro. Sans cette optimisation matérielle, le projet n'aurait pas été viable.

3.2.2 Le problème du son : recouvrement audio et gestion de l'écho

Le recouvrement audio, c'est-à-dire la situation où Pepper parle pendant que le client parle, constitue le problème technique numéro un identifié dans les recherches. Quand Pepper parle, ses microphones captent sa propre voix. Si ce signal est envoyé au système de Speech to Speech dans le cloud, l'IA interprète la voix de Pepper comme celle du client, ce qui génère des réponses incohérentes ou des boucles de conversation.

Deux stratégies techniques existent pour traiter ce problème, correspondant à deux architectures différentes :

- **Stratégie demi-duplex : remplacement par zéros.** La solution documentée dans le projet d'Osnabrück consiste à remplacer le flux audio envoyé au service cloud par des zéros lorsque Pepper parle. Cette approche est très robuste et rapide à stabiliser. Elle réduit fortement le risque de boucle audio et de comportements incohérents. La limite assumée est l'absence de *barge-in* vocal : le client ne peut pas interrompre Pepper en parlant. Dans le cadre d'un usage magasin avec des réponses courtes et structurées, cette limite reste acceptable.
- **Stratégie plein duplex : annulation d'écho acoustique (AEC).** Le plein duplex nécessite de continuer à écouter pendant que Pepper parle, tout en évitant que la voix de Pepper ne soit renvoyée au modèle comme si elle provenait du client. La brique de base est l'annulation d'écho acoustique. Le principe est le suivant : le micro capte un mélange constitué de la voix du client plus l'écho de ce qui sort des haut-parleurs. L'AEC utilise comme référence le signal exact envoyé aux haut-parleurs (signal "play") et le signal micro (signal "rec"), puis produit un signal de sortie ("out") où l'écho est fortement réduit. L'API SpeexDSP décrit explicitement ce triplet rec/play/out et le rôle de l'écho.

La synchronisation constitue une difficulté réelle. L'écho n'arrive pas instantanément : il existe un délai entre le moment où le son sort du haut-parleur, se propage dans l'air, et arrive au micro. Ce délai varie avec la salle et la position. Certaines implémentations modernes, notamment l'écosystème WebRTC, proposent des modes "delay-agnostic" qui s'appuient sur une estimation interne du délai pour gérer le cas où ce délai n'est pas parfaitement connu.

L'intégration de l'AEC nécessite de conserver en permanence une copie horodatée du signal envoyé à Pepper comme référence, puis d'appliquer l'AEC sur le Mac, soit avant le beamforming (AEC par canal puis combinaison), soit après (beamforming puis AEC en mono). Le résultat attendu si l'AEC est correctement réglée est que le flux envoyé à OpenAI pendant que Pepper parle contient surtout la voix du client et beaucoup moins la voix de Pepper. Le *barge-in* devient alors utilisable : le client peut couper la parole et la réponse peut être interrompue.

Contraintes de format audio identifiées

La documentation NAOqi indique que Pepper fournit un flux micro en 4 canaux à 48 kHz en format par défaut, ou en 1 canal à 16 kHz selon les préférences. L'enregistrement micro en WAV se fait en 16 bits, 48 kHz, 4 canaux, et il est possible de rejouer un fichier WAV comme entrée micro via `setFileAsInput` au format attendu (48 kHz, 16 bits, 4 canaux interleavés).

Pour la sortie audio, l'utilisation du module standard `ALAudioPlayer` (conçu pour lire des fichiers MP3/WAV) est écartée car elle introduit une latence d'écriture/lecture disque incompatible avec le temps réel. Nous retenons l'utilisation de `ALAudioDevice`, une API plus bas niveau qui permet le streaming direct. La méthode `sendRemoteBufferToOutput` accepte des buffers PCM bruts (16 bits, 48 kHz, stéréo interleavé, taille maximale 16384 frames). Cette approche permet d'injecter directement les chunks audio décodés dans la carte son du robot sans passer par un fichier temporaire.

La documentation OpenAI Realtime précise que le format audio attendu en audio/pcm est 24 kHz, mono, PCM. Le transport se fait via WebSocket ou WebRTC. L'audio est envoyé par événements `input_audio_buffer.append` et la sortie audio arrive sous forme de deltas `response.output_audio.delta`.

Ces contraintes imposent une chaîne de traitement complète : réception du flux Pepper 4 canaux 48 kHz, beamforming orienté vers l'avant pour privilégier la source en face de Pepper, réduction de bruit modérée puis contrôle de gain léger, passage en mono, rééchantillonnage en 24 kHz format PCM16, envoi à OpenAI Realtime via WebSocket, réception de l'audio de réponse en flux, décodage, rééchantillonnage vers 48 kHz, duplication stéréo, puis envoi à Pepper via `sendRemoteBufferToOutput`.

Constitution d'un jeu d'essai audio rejouable

L'accès au robot étant limité, la nécessité de constituer un corpus audio réutilisable s'est imposée rapidement. Le format le plus utile identifié est le WAV 48 kHz, 16 bits, 4 canaux depuis Pepper, format explicitement documenté.

Les enregistrements doivent couvrir des situations représentatives : client face à Pepper distance stable, client qui bouge latéralement ou en profondeur, bruit de fond faible puis réaliste, cas de recouvrement où Pepper parle puis le client coupe la parole, cas inverse où le client parle puis Pepper répond avec bruit constant.

Pour préparer l'AEC en vue du plein duplex, il est également nécessaire d'archiver la référence, c'est-à-dire le signal audio réellement envoyé à Pepper au moment où Pepper parlait. Cette référence est indispensable pour estimer et supprimer l'écho.

Deux mécanismes de relecture ont été identifiés : relecture sur le Mac avec un outil qui lit le WAV 4 canaux et rejoue les trames au même rythme que le temps réel en alimentant la chaîne beamforming, filtrage, rééchantillonnage puis envoi vers OpenAI, et relecture sur Pepper via `setFileAsInput` qui

permet de remplacer l'entrée micro par un fichier WAV pour tester un module NAOqi comme s'il recevait un flux micro réel.

3.3 Convergence des contraintes

L'architecture finale résulte d'un empilement de contraintes interdépendantes. La latence perçue doit rester sous 1,2 seconde. Cette exigence impose un traitement local pour certaines tâches plutôt qu'un envoi systématique dans le cloud. Le traitement local demande une optimisation matérielle sérieuse, d'où le choix de MLX. MLX impose une compression des modèles en 4-bit pour qu'ils tiennent en mémoire, 5GB au lieu de 14GB. La compression réduit légèrement la précision du modèle de vision, ce qui nécessite une confirmation quand le système hésite et un fallback vers la lecture de code-barres en cas d'échec. Le code-barres rallonge l'interaction, donc la latence doit être masquée avec des micro-phrases et des animations.

Le recouvrement audio impose un choix d'architecture : demi-duplex avec remplacement par zéros pour une version stable rapidement, ou plein duplex avec AEC pour permettre le *barge-in* au prix d'une complexité technique supérieure et d'une synchronisation délicate. La première stratégie offre une robustesse immédiate, la seconde demande une phase de mise au point progressive avec constitution d'un corpus audio de test.

Le RGPD interdit la reconnaissance faciale. Les utilisateurs attendent pourtant que le robot se souvienne d'eux quand ils reviennent avec un autre produit deux minutes plus tard. Un système de vecteur basé sur la silhouette associé à un vecteur vocal a été conçu, stocké en mémoire vive avec effacement automatique après sept minutes. Cette solution fonctionne dans la majorité des cas avec quelques erreurs acceptables.

L'agent conversationnel dans le cloud fluidifie le dialogue mais expose à un risque : les questions médicales. Deux protections ont été mises en place. Le LLM reçoit des instructions explicites pour refuser ce type de questions. En parallèle, un filtre local détecte les mots-clés dangereux dans la transcription texte fournie par OpenAI Realtime.

Chaque solution technique génère de nouveaux problèmes nécessitant de nouvelles solutions. L'architecture paraît complexe mais vise un objectif simple : faire répondre Pepper de façon crédible en moins de 1,2 seconde tout en gérant proprement le son dans un environnement bruité.

3.4 Définition d'un périmètre réaliste

Les semaines de recherche ont progressivement réduit le périmètre initial. Le projet visait initialement toute la parapharmacie, il s'est recentré sur le rayon capillaire. La reconnaissance faciale envisagée pour personnaliser l'expérience a été abandonnée face aux contraintes RGPD. L'idée de tout traiter en local sur Pepper s'est heurtée aux limitations du robot et a conduit vers une architecture hybride.

Le choix entre demi-duplex et plein duplex pour la gestion audio s'est posé clairement. La stratégie demi-duplex offre une voie de livraison rapide et stable. La stratégie plein duplex avec AEC représente une amélioration ultérieure possible si le temps le permet et si la constitution du corpus audio de test est suffisante. Ces restrictions relèvent du pragmatisme. Trente produits, trois semaines de développement effectif, un accès à Pepper sur rendez-vous seulement. Dans ce contexte, un petit périmètre fonctionnel vaut mieux qu'un projet ambitieux qui échoue.

Ce périmètre réaliste suppose que nous puissions tester et améliorer le projet lors de sa conception. Le cadre méthodologique permettant ce développement itératif tout en respectant les contraintes RGPD est détaillé en section 2.4.4 (Cadre de test et développement)

4. Architecture proposée

4.1 Vue d'ensemble

L'architecture repose sur une séparation stricte des rôles entre trois composants principaux :

- **Pepper (Interface sensorielle et expressive)** assure la capture via capteurs (caméras pour image produits, microphones pour audio utilisateur, capteur 3D pour détection présence/départ) et « une approche d'expression humaine » via des actionneurs (tablette tactile pour affichage info, haut-parleurs pour voix, moteurs pour gestes, LEDs pour états visuels). Son rôle est la capture sensorielle et l'expressivité robotique, sans traitement complexe embarqué.
- **Mac M4 Pro (Cerveau local)** gère la vision (VLM local MLX pour l'identification d'un produit), les données (base produits locale SQLite 30 références), l'orchestration (state machine coordonnant Pepper + Agent cloud), et fait office de pont réseau entre Pepper (RJ45 local) et services cloud.
- **Agent cloud (Intelligence conversationnelle)** utilise un LLM (OpenAI GPT Realtime API *speech-to-speech*) pour la gestion du contexte (multi-tours conversationnels avec une mémoire court-terme) et le streaming (génération audio en flux continu).

La connexion réseau privilégie Ethernet RJ45 entre Pepper et Mac (latence <5ms, stable), tandis que la connexion Mac-Cloud utilise Internet (latence 50-200ms variable, gérée).

De manière pragmatique, la configuration réseau idéale serait d'utiliser deux adaptateurs RJ45 sur le mac avec deux plans d'adressage IP différents, un avec le robot pour l'adaptateur 1, l'autre pour accéder à internet pour l'adaptateur 2. Notre objectif étant de ne pas réintroduire de la latence artificielle avec du wifi.

4.2 Technologies clés et justifications

4.2.1 Vision locale (VLM MLX)

Le modèle sélectionné est Qwen2-VL-7B-Instruct quantisé 4-bit. Ce choix repose sur plusieurs critères : performances SOTA (State-Of-The-Art) sur benchmarks vision-langage, latence de 200-350ms sur Mac M4 Pro avec MLX (contre 2000-3000ms PyTorch CPU), empreinte mémoire réduite à environ 5GB RAM avec quantization 4-bit (vs 14GB FP16), et support multilingue natif du français.

Une alternative est possible chez mistral. les deux solutions seront évaluées.

Le pipeline technique comprend la capture d'image en résolution 640 × 480 via caméra Pepper frontale (NAOqi ALVideoDevice), un pré-processing (resize 448 × 448, normalisation), l'inférence MLX, et un post-processing (parsing réponse LLM, extraction nom produit + confiance).

La décision repose sur des seuils : au-dessus de 85%, affichage direct. Entre 60-85%, Top-3 avec confirmation, en dessous de 60%, fallback code-barres.

Trois optimisations techniques permettent de réduire la latence du VLM :

- **Réduction de la taille du modèle (quantization) :** Le modèle Qwen2-VL-7B est compressé en format 4-bit au lieu de 16-bit, réduisant son empreinte mémoire de 14GB à 5GB. Cette compression n'affecte pas significativement la qualité de reconnaissance avec une perte <2% sur benchmarks standards (source huggingface) tout en accélérant les calculs.

- **Réutilisation des calculs (cache de prompts) :** Lorsque le même prompt est utilisé plusieurs fois ("Identifie ce produit capillaire"), le système mémorise une partie des calculs intermédiaires, évitant de les refaire intégralement. Gain : 50-100ms par requête répétitive.
- **Traitement groupé d'images :** Si plusieurs photos du même produit sont capturées rapidement (3 frames en rafale), le système les traite ensemble au lieu de séquentiellement, réduisant la latence moyenne de 400ms à 150ms par image.

4.2.2 Conversation speech-to-speech et gestion du recouvrement audio

L'architecture utilise OpenAI Realtime API (GPT avec voice) via WebSocket bidirectionnel. Le pipeline audio suit ce flux : Pepper capture via 4 micros directionnels (format 48kHz 16-bit, NAOqi ALAudioDevice), le Mac effectue le resampling (48kHz → 24kHz pour être compatible avec le format d'OpenAI), le streaming Mac Cloud envoie des chunks de 100ms encodés en base64 via WebSocket asynchrone, la réception Cloud Mac reçoit des chunks audio dès génération première syllabe, et le playback Mac Pepper via streaming direct. Le Mac utilise l'API ALAudioDevice pour envoyer les buffers PCM décodés directement à la carte son de Pepper via la fonction `sendRemoteBufferToOutput`. Cette méthode supprime totalement la latence liée aux fichiers intermédiaires (pas de conversion WAV, pas de serveur HTTP). Le délai entre la réception du paquet réseau et la vibration du haut-parleur est ainsi réduit au strict minimum (buffer matériel).

Le problème du recouvrement audio

Le recouvrement audio constitue le problème technique numéro un de cette architecture. Quand Pepper parle, ses quatre microphones captent inévitablement sa propre voix. Si ce signal est envoyé tel quel au service cloud OpenAI Realtime, l'IA interprète la voix de Pepper comme celle du client. Cela génère des réponses incohérentes ou déclenche des boucles de conversation où Pepper répond à ses propres propos. Les recherches menées sur les projets académiques existants (notamment le dépôt d'Osnabrück "pepper_speech_recognition") confirment que ce problème apparaît systématiquement dans les déploiements réels et constitue un blocage critique si non traité.

Deux stratégies techniques identifiées

Deux approches existent pour traiter le recouvrement audio, correspondant à deux architectures différentes avec des compromis distincts.

- Stratégie 1 : Demi-duplex avec remplacement par zéros (choix V1)

Cette stratégie, documentée dans le projet d'Osnabrück, consiste à remplacer le flux audio envoyé au service cloud par des zéros lorsque Pepper parle. Techniquement, le système détecte quand Pepper est en train de restituer de l'audio (état "parole active") et durant cette période, le buffer audio capté par les microphones est remplacé par un signal silence (zéros) avant envoi à OpenAI Realtime.

Les avantages de cette approche sont multiples. Elle est très robuste et rapide à stabiliser. Elle réduit fortement le risque de boucle audio et de comportements incohérents. L'implémentation est simple : un flag binaire "Pepper parle" suffit à activer ou désactiver le remplacement. Les tests peuvent se concentrer sur la qualité conversationnelle plutôt que sur des problèmes de traitement du signal complexes.

La limite assumée est l'absence de barge-in vocal : le client ne peut pas interrompre Pepper en parlant. Dans le cadre d'un usage magasin avec des réponses courtes et structurées explicitement demandées au modèle, cette limite reste acceptable. L'utilisateur peut toujours utiliser la tablette tactile pour interrompre ou relancer une interaction.

Cette stratégie est retenue pour la version livrable minimale du démonstrateur. Elle permet d'obtenir rapidement un système stable et démontrable, validant l'architecture globale avant d'envisager des améliorations plus complexes.

- Stratégie 2 : Plein duplex avec annulation d'écho acoustique (amélioration future potentielle)

Le plein duplex nécessite de continuer à écouter pendant que Pepper parle, tout en évitant que la voix de Pepper ne soit renvoyée au modèle comme si elle provenait du client. La brique de base est l'annulation d'écho acoustique (Acoustic Echo Cancellation, AEC). Le principe est le suivant : le micro capte un mélange constitué de la voix du client plus l'écho de ce qui sort des haut-parleurs. L'AEC utilise comme référence le signal exact envoyé aux haut-parleurs (signal "play") et le signal micro (signal "rec"), puis produit un signal de sortie ("out") où l'écho est fortement réduit. L'API SpeexDSP décrit explicitement ce triplet rec/play/out et le rôle de l'écho.

La synchronisation constitue une difficulté réelle. L'écho n'arrive pas instantanément : il existe un délai entre le moment où le son sort du haut-parleur, se propage dans l'air, et arrive au micro. Ce délai varie avec la salle et la position. Certaines implémentations modernes, notamment l'écosystème WebRTC, proposent des modes "delay-agnostic" qui s'appuient sur une estimation interne du délai pour gérer le cas où ce délai n'est pas parfaitement connu.

L'intégration de l'AEC nécessiterait de conserver en permanence une copie horodatée du signal envoyé à Pepper comme référence, puis d'appliquer l'AEC sur le Mac, soit avant le beamforming (AEC par canal puis combinaison), soit après (beamforming puis AEC en mono).

Le résultat attendu si l'AEC est correctement réglée est que le flux envoyé à OpenAI pendant que Pepper parle contient surtout la voix du client et beaucoup moins la voix de Pepper. Le barge-in devient alors utilisable : le client peut couper la parole et la réponse peut être interrompue via la fonction `interrupt_response` de l'API OpenAI Realtime.

Cette stratégie représente une amélioration possible si le temps le permet et si la constitution du corpus audio de test est suffisante. Elle n'est pas dans le périmètre strict de la version 1 mais fait partie des perspectives d'évolution identifiées immédiatement applicables suivant les résultats obtenus.

Chaîne de traitement audio complète

Contraintes audio Pepper (latences et trames)

Le flux micro est reçu sous forme de blocs (buffering interne). Cette granularité impose une latence minimale (ordre de grandeur ~170 ms en 4ch/48 kHz, annoncée dans la documentation officielle de Pepper). Les conversions (4ch→mono, 48k→24k), la synchronisation et l'envoi cloud s'empilent sur ce plancher.

La chaîne de traitement audio mise en œuvre sur le Mac pour la version 1 suit les étapes suivantes :

1. Réception du flux Pepper 4 canaux 48 kHz via la connexion réseau RJ45.
2. Beamforming orienté vers l'avant pour privilégier la source sonore en face de Pepper. Cette technique exploite les quatre microphones pour former un faisceau directionnel, réduisant le bruit ambiant latéral et arrière.
3. Réduction de bruit modérée puis contrôle de gain léger. L'objectif n'est pas de "nettoyer à blanc" mais d'améliorer l'intelligibilité tout en conservant une voix naturelle. Un traitement trop agressif rendrait la voix artificielle et dégraderait paradoxalement la reconnaissance.
4. Passage en mono et rééchantillonnage en 24 kHz format PCM16 pour correspondre aux exigences de l'API OpenAI Realtime.
5. Application de la stratégie demi-duplex : si Pepper est en train de parler (flag actif), remplacement du buffer audio par des zéros avant envoi.
6. Envoi à OpenAI Realtime via WebSocket avec des événements `input_audio_buffer.append`.
7. Réception de l'audio de réponse en flux sous forme de deltas `response.output_audio.delta` dès génération de la première syllabe.
8. Décodage, rééchantillonnage vers 48 kHz, duplication stéréo.
9. Envoi à Pepper via `sendRemoteBufferToOutput`.

Configuration VAD et gestion du contexte

La configuration VAD (Voice Activity Detection) d'OpenAI Realtime permet un réglage fin avec des paramètres comme threshold (sensibilité détection voix, 0.5 = équilibre bruit/silence) et silence_duration_ms (durée silence pour considérer fin phrase, 500ms = détection sémantique). Ces paramètres seront ajustés empiriquement lors des tests terrain pour trouver le meilleur compromis entre réactivité et robustesse au bruit ambiant.

La gestion du contexte multi-tours injecte le produit actuel dans le prompt système : *"Tu es assistant parapharmacie. Produit actuel : L'Oréal Elsève Hydratation (shampooing cheveux normaux, 9.90€). Réponds questions sur CE produit uniquement. Redirige questions médicales vers pharmacien."* Cette injection dynamique permet au LLM de maintenir une cohérence conversationnelle sur plusieurs échanges successifs sans re-scanner le produit.

Constitution d'un corpus audio de test

L'accès au robot étant limité (disponible au laboratoire uniquement sur rendez-vous), la constitution d'un corpus audio réutilisable est indispensable. Le format retenu est le WAV 48 kHz, 16 bits, 4 canaux depuis Pepper, format explicitement documenté par NAOqi.

Les enregistrements doivent couvrir des situations représentatives variées pour éviter d'optimiser uniquement sur un cas de laboratoire idéal :

- Client face à Pepper, distance stable (cas nominal).
- Client qui bouge latéralement ou en profondeur (variation de direction source).
- Bruit de fond faible puis bruit de fond réaliste (fond sonore magasin, voix environnantes).
- Cas de recouvrement : Pepper parle puis le client coupe la parole (test stratégie demi-duplex).
- Cas inverse : le client parle puis Pepper répond avec bruit constant (test VAD sémantique).

Deux mécanismes de relecture ont été identifiés. Relecture sur le Mac : un outil lit le WAV 4 canaux et rejoue les trames au même rythme que le temps réel en alimentant la chaîne beamforming, filtrage, rééchantillonnage puis envoi vers OpenAI. Relecture sur Pepper : la documentation NAOqi mentionne la possibilité de remplacer l'entrée micro par un fichier WAV via setFileAsInput (48 kHz, 16 bits, 4 canaux interleavés), permettant de tester un module NAOqi comme s'il recevait un flux micro réel. Cette approche de test sur corpus enregistré permet de développer et déboguer sur le Mac sans dépendre du robot au quotidien, compensant la contrainte d'accès intermittent à Pepper.

4.2.3 Base produits

Le schéma SQLite pour 30 produits V1 comprend les champs : id, ean (code-barres EAN-13), nom, marque, categorie (shampooing/après-shampooing/masque), type_cheveux (normaux/secs/gras/colorés), prix, volume, usage, precautions, photo_url, et timestamp_creation. Des index sont créés sur ean, categorie et type_cheveux pour optimiser les requêtes.

La population de la base utilise un scraping semi-automatique : un script BeautifulSoup extrait les données d'un site e-commerce parapharmacie (nom, marque, prix, description), suivi d'une complétion manuelle pour la catégorisation, l'usage et les précautions (on se limite à 30 produits), et le téléchargement d'images haute résolution des produits.

4.2.4 Orchestrateur (State Machine)

Les états système incluent : IDLE (veille, attend présence), GREETING (accueil utilisateur), AWAITING_INTENT (attend choix "produit" ou "conseil"), SCANNING_PRODUCT (analyse visuelle produit), CONFIRMING_TOP3 (attente confirmation Top-3), SCANNING_BARCODE (mode fallback

code-barres), `DISPLAYING_INFO` (fiche produit affichée), `CONVERSING` (conversation multi-tours active), `COMPARING` (comparaison multi-produits), `ADVISING` (conseil capillaire sans produit), et `ENDING` (fin session).

Les transitions principales suivent une logique claire : `IDLE` détecte une présence et passe à `GREETING`, qui détecte la voix et passe à `AWAITING_INTENT`. Selon l'intent détecté (produit ou conseil), le système bascule vers `SCANNING_PRODUCT` ou `ADVISING`. Le scan produit mène à `DISPLAYING_INFO` (confiance haute), `CONFIRMING_TOP3` (confiance moyenne), ou `SCANNING_BARCODE` (confiance basse). Une fois l'info affichée, les questions déclenchent `CONVERSING`, qui peut revenir à `SCANNING_PRODUCT` si un nouveau produit est présenté, ou passer à `ENDING` si l'utilisateur part.

La gestion asynchrone utilise `asyncio` pour gérer plusieurs tâches parallèles : `monitor_presence()` surveille le capteur 3D en continu, `process_video_stream()` traite les frames caméra si nécessaire, `handle_audio_stream()` gère l'audio bidirectionnel en conversation, `update_tablet()` rafraîchit l'UI, et `trigger_animations()` synchronise gestes et LEDs Pepper. La coordination s'effectue via des queues `asyncio`.

4.3 Sécurité et gestion hors périmètre

4.3.1 Détection produits médicaments

Une liste noire de 50-100 codes-barres EAN-13 de médicaments courants en parapharmacie (Doliprane, Advil, antibiotiques sans ordonnance) est pré-chargée. Lors du scan barcode, si un EAN de médicament est détecté, Pepper affiche des LEDs orange (exception sécurité, seul usage couleur chaude), effectue un geste "stop" (mains levées), dit "Attention, médicament détecté. Consultez le pharmacien pour ce produit", pointe la direction du comptoir pharmacie, et l'événement est loggé pour traçabilité sécurité.

4.3.2 Détection questions médicales

En mode speech-to-speech, l'audio utilisateur est envoyé directement à l'agent OpenAI qui génère une réponse audio. Pour des raisons de sécurité (éviter les réponses médicales) et de traçabilité durant la phase de test (analyser les questions posées pour améliorer le système), nous devons pouvoir filtrer certaines questions et conserver une trace textuelle des échanges.

L'API OpenAI Realtime fournit simultanément deux éléments : la transcription textuelle de la question posée et la réponse audio générée. Le système exploite cette transcription pour mettre en place un doublement de la protection :

Première couche : instructions au modèle

Le prompt système précise explicitement : "Si l'utilisateur pose une question médicale (diagnostic, traitement, interaction médicamenteuse, symptômes), tu dois répondre UNIQUEMENT : 'Cette question dépasse mes compétences. Je vous invite à demander conseil au pharmacien.' Ne développe jamais de réponse médicale."

Cette instruction constitue la première ligne de défense et gère les cas où le filtrage local échouerait. On part du principe que le LLM peut « halluciner ».

Seconde couche : filtrage local par mots-clés

Dès réception de la transcription, le Mac vérifie la présence de termes sensibles pré-définis : "douleur", "symptôme", "traitement", "ordonnance", "diagnostic", "allergie", "effets secondaires", "contre-indication", "infection", "maladie".

Si un mot-clé est détecté :

- Arrêt immédiat du stream audio réponse LLM (si déjà démarré).
- Réponse pré-générée (voix OpenAI, <50ms lecture locale) : 'Cette question dépasse mes compétences. Consultez le pharmacien.' Cette phrase est générée une fois au démarrage du système avec la même voix que l'accueil, puis stockée localement. En cas de détection keyword médical, elle est jouée immédiatement, garantissant cohérence vocale et réactivité. La voix TTS native de Pepper n'est utilisée qu'en mode dégradé (perte connexion réseau).
- Logging de l'événement pour traçabilité en pré-phase de test.

Si aucun mot-clé n'est détecté, la réponse LLM se joue normalement.

Avantages de cette approche double couche :

- **Rapidité** : le filtrage local réagit en 50ms contre 300ms+ pour la génération LLM complète.
- **Robustesse** : si le LLM "oublie" l'instruction ou l'interprète mal, le filtre local rattrape l'erreur.
- **Sécurité renforcée** : aucune réponse médicale ne peut être diffusée, même en cas de défaillance d'une des deux couches.
- **Limite assumée** : La détection par mots-clés est une approche simple qui peut générer des faux positifs (exemple : "sans douleur" détecte "douleur"). Cette limite est acceptable pour un démonstrateur V1 et pourrait être améliorée par une classification machine learning si l'on dépassait la phase de démonstrateur.

Produits hors rayon capillaire

Avant d'identifier un produit précisément, le système vérifie qu'il s'agit bien d'un produit capillaire. Le VLM reçoit d'abord un prompt de classification : "Ce produit visible est-il un produit capillaire (shampooing, après-shampooing, masque, soin cheveux) ? Réponds OUI ou NON."

Si le VLM retourne "NON" (produit détecté comme dentifrice, crème visage, maquillage, etc.), Pepper dit : "Je détecte un produit hors rayon capillaire. Je suis spécialisé shampoings et soins cheveux uniquement" et retourne à l'état IDLE sans tenter d'identification.

Si le VLM retourne "OUI", le système procède à l'identification précise du produit (prompt détaillé avec nom et marque), puis recherche dans la base capillaire. Si le produit capillaire n'est pas trouvé dans la base, le fallback code-barres est proposé.

Cette approche en deux étapes (classification puis identification) permet de rejeter proprement les produits hors périmètre sans nécessiter une base exhaustive de tous les produits non-capillaires existants.

5. Conception fonctionnelle

5.1 Accueil et choix d'interaction

5.1.1 Détection présence et accueil proactif

Le capteur 3D Pepper (Asus Xtion PRO LIVE) scanne l'environnement frontal avec une portée de 0.5-3m et un angle d'environ 60° . Il détecte une silhouette humaine via la carte de profondeur (depth map).

Lorsqu'une silhouette est détectée à une distance inférieure à 2m et reste stable pendant 1 seconde, Pepper effectue une rotation du corps et de la tête vers la silhouette (animation fluide sur 1s), les LEDs des yeux passent de violet (idle) à bleu (écoute).

Le système calcule immédiatement le vecteur silhouette de la personne détectée et vérifie s'il correspond à une session en RAM avec TTL valide. Si correspondance potentielle trouvée, Pepper dit : *"Puis-je vous aider ?"* et attend la première parole pour calculer le vecteur vocal et confirmer l'identité.

Si aucune correspondance ou si la signature vectorielle ne correspond pas après vérification vocale, Pepper joue l'accueil standard : *"Bonjour, je peux vous aider. Produit ou question générale ?"*. Une animation de geste main ouvert (invitation) accompagne l'accueil.

Simultanément, la tablette affiche deux boutons tactiles grande taille :

- **[PRODUIT]** : "Je veux des infos sur un produit".
- **[QUESTION GÉNÉRALE]** : "J'ai besoin de conseils".

Cette formulation claire (vocale et visuelle) propose explicitement deux modalités d'interaction sans présupposer le besoin de l'utilisateur. Le système active immédiatement la connexion OpenAI Realtime speech-to-speech et attend l'une des actions suivantes : réponse vocale, clic tablette, ou présentation produit (geste détecté).

5.1.2 Choix d'interaction : produit physique ou conseil général

Modalité A : L'utilisateur choisit "Produit"

L'utilisateur indique qu'il souhaite des détails sur un produit spécifique. Trois déclencheurs possibles détectent cette intention :

- **Réponse vocale directe** : L'utilisateur répond "Produit" ou variantes ("un produit", "j'ai un produit", "shampooing"). Le LLM speech-to-speech détecte l'intent "produit" dans la transcription audio.
- **Clic tablette** : L'utilisateur clique sur le bouton tactile **[PRODUIT]** affiché sur la tablette.
- **Présentation produit directe** : Le capteur 3D détecte un mouvement d'approche rapide vers la caméra (objet se rapproche, distance diminue de plus de 30cm en moins de 2 secondes), même si l'utilisateur n'a pas répondu vocalement ni cliqué tablette. Le système interprète ce geste comme une intention de présentation produit.

Dès détection de l'un de ces trois déclencheurs, Pepper confirme et guide (streaming audio OpenAI Realtime) : *"Très bien, approchez votre produit bien face à la caméra"* avec une animation pointant sa caméra frontale (geste pédagogique). Les LEDs passent de bleu à vert (passage en mode analyse).

Simultanément, la tablette affiche une aide visuelle : image schématique de la caméra Pepper avec flèche et texte *"Présentez le produit ici"*. Le système lance le workflow d'identification VLM.

Modalité B : L'utilisateur choisit "Question générale"

L'utilisateur indique qu'il souhaite un conseil sans produit spécifique en main. Trois déclencheurs possibles détectent cette intention :

- **Réponse vocale directe** : L'utilisateur répond "Question" ou variantes ("question générale", "conseil", "j'ai une question"). Le LLM speech-to-speech détecte l'intent "question_generale" dans la transcription.
- **Clic tablette** : L'utilisateur clique sur le bouton tactile **[QUESTION GÉNÉRALE]** affiché sur la tablette.
- **Question directe sans attendre** : L'utilisateur pose directement une question précise sans passer par le choix ("Quel shampoing pour cheveux secs?", "Vous avez des soins pour cheveux colorés ?", "Je cherche un masque hydratant"). Le LLM détecte l'intent "conseil_general" caractérisé par l'absence de référence à un produit spécifique tenu et la présence de mots-clés comme "quel", "chercher", "recommander", "choisir", "conseiller".

Dès détection de l'un de ces trois déclencheurs, Pepper répond (streaming audio OpenAI Realtime) : *"Je suis à votre écoute pour vos questions sur les produits capillaires."* Le système bascule en mode ADVISING (conseil capillaire sans produit).

Selon la précision de la question ou du choix :

- **Intent clair et spécifique** ("shampoing cheveux secs" dit vocalement) : affichage direct d'une liste de produits filtrés sur la tablette avec la sélection correspondante.
- **Intent vague ou choix sans précision** (clic bouton **[QUESTION]** sans question vocale, ou vocal "question" sans préciser) : Pepper pose une question de précision *"Quel est votre type de cheveux ? Secs, gras, normaux, colorés ?"*. L'utilisateur répond vocalement.

Gestion des cas ambigus

- **Actions simultanées produit + question** : Si l'utilisateur clique **[PRODUIT]** sur la tablette ET dit simultanément une question vocale, le système donne priorité au clic tablette (action tactile explicite et intentionnelle). Si l'utilisateur présente physiquement un produit devant la caméra (geste détecté capteur 3D) pendant qu'il dit "question", le système donne priorité au produit visible (objet concret immédiat présent). Raisonnement : le produit physique ou le clic tablette constituent des actions concrètes immédiates, la question peut être reformulée ou précisée après traitement de l'action prioritaire.
- **Hésitation ou silence prolongé** : Si l'utilisateur ne réalise aucune action (ni réponse vocale, ni clic tablette, ni présentation produit) pendant 10 secondes après l'accueil, Pepper relance avec une formulation différente pour éviter la simple répétition : *"Vous pouvez me montrer un produit, me poser une question, ou utiliser la tablette."* Les boutons tactiles restent affichés et bien visibles. Si toujours aucune action après 30 secondes au total, le système retourne à l'état IDLE.
- **Changement de modalité en cours d'interaction** : Si l'utilisateur est actuellement en mode ADVISING (conseil général, navigation tablette active affichant liste produits filtrés) et présente soudain un produit physique devant la caméra, le système détecte le geste de présentation mais demande confirmation avant de basculer : *"Vous voulez que je regarde ce produit?"*. Il attend une réponse vocale ("oui") ou un clic tablette de confirmation. Cette

précaution évite de déclencher un scan VLM accidentel si l'utilisateur manipule simplement un produit sans intention de le faire identifier.

5.1.3 Convention LEDs et expressivité continue

Les guidelines B2BD (*Business to Business Development*) de SoftBank insistent sur la nécessité absolue que Pepper "reste vivant" pendant toute l'interaction. L'état idle ne signifie en aucun cas immobilité figée : des micro-mouvements de tête simulent une respiration naturelle, les LEDs varient avec un pulse lent (variation d'intensité subtile), et le regard (orientation tête) balaye l'environnement de façon naturelle.

La convention LEDs suit une logique claire et cohérente tout au long de l'interaction :

- **Bleu** : Écoute active (capture audio utilisateur en cours).
- **Vert** : Analyse ou traitement (inférence VLM pour identification produit, ou traitement LLM pour génération réponse).
- **Blanc** : Parole (TTS Pepper en train de parler, restitution audio).
- **Orange** : Alerte sécurité (médicament détecté ou question médicale détectée, unique exception à l'usage de couleur chaude).
- **Violet** : Veille (état idle, attente de présence utilisateur).
- **Rouge** : Jamais utilisé (banni car perçu comme signal de colère ou d'erreur selon les guidelines B2BD).

Les transitions entre couleurs sont toujours progressives (fade de 300ms) pour éviter un clignotement brutal qui pourrait être perçu comme agressif ou défaillance technique.

5.2 Workflow identification produit (mode SCANNING)

5.2.1 Déclenchement du scan visuel

Une fois que l'utilisateur a choisi la modalité "Produit", Pepper guide vocalement : *"Très bien, montrez-moi le produit."* Les LEDs passent de bleu (écoute) à vert (analyse). La tablette affiche un message simple : *"Présentez le produit face à la caméra."*

Le système joue immédiatement une micro-phrase locale pré-enregistrée : *"Je regarde"* (voix OpenAI Realtime générée au démarrage, stockée en WAV, lecture inférieure à 50ms). Cette réaction audio instantanée masque la latence de capture et d'analyse qui suit.

Gestion du mouvement et stratégie de capture

Pendant que la micro-phrase joue ("Je regarde"), le système lance la capture.

- **Hypothèse terrain (assumée)** : L'utilisateur peut bouger pendant la présentation du produit. Notre choix de capturer plusieurs images vise justement à absorber cette variabilité sans imposer un cadrage strict ni traitement "anti-flou" coûteux.
- **Choix V1** : Nous comparerons deux stratégies d'acquisition à budget de temps constant, puis nous retiendrons celle qui maximise le taux d'identification :
 - **(A) Rafale courte** : Plusieurs frames sur ~200–300 ms (minimise le mouvement entre frames).
 - **(B) Rafale espacée** : 3 frames sur ~600–700 ms (maximise la diversité d'angles).
- **Objectif** : Améliorer la robustesse sans ajouter de calculs de qualité d'image coûteux et sans dégrader la latence.

5.2.2 Analyse parallèle : VLM et code-barres

Les trois images capturées sont analysées simultanément par deux processus indépendants lancés en parallèle.

Processus 1 : Détection code-barres

La bibliothèque pyzbar analyse les trois images pour détecter un éventuel code-barres EAN-13 (traitement environ 100ms total). Si un code-barres identique est détecté dans au moins deux des trois images, le résultat est considéré fiable.

Processus 2 : Analyse VLM

Le modèle VLM (Qwen2-VL-7B quantisé 4-bit via MLX) analyse les trois images accompagnées de la liste des 30 produits de référence et retourne les trois produits les plus probables avec leurs scores de confiance (traitement environ 300-400ms). Le modèle VLM analyse les trois images en un seul appel d'inférence, accompagnées de la liste des 30 produits de référence.

Arbitrage entre les deux résultats

Le système attend que les deux processus se terminent (latence totale = $\max(100\text{ms pyzbar}, 350\text{ms VLM}) \approx 350\text{ms}$), puis décide quelle méthode utiliser selon une logique de priorité.

- **Priorité 1 : Code-barres détecté de façon fiable.** Si pyzbar a détecté un code-barres identique dans au moins deux des trois images, ce résultat est utilisé en priorité car déterministe. Le code-barres est vérifié contre la liste noire médicaments puis recherché dans la base SQLite. Si trouvé, Pepper annonce : *"Code-barres identifié : [nom produit]"* et la fiche produit s'affiche. Le résultat VLM est ignoré.
- **Priorité 2 : VLM si code-barres non détecté ou incohérent.** Si aucun code-barres n'a été détecté ou si les trois images donnent des codes-barres différents (incohérence), le système utilise le résultat VLM. Trois cas selon le score de confiance du premier produit :
 - **Confiance haute ($\geq 85\%$) :** Pepper annonce *"J'ai identifié [nom produit]"*, affichage fiche complète, passage en mode CONVERSING.
 - **Confiance moyenne (60-85%) :** Affichage Top-3 sous forme de cartes numérotées. Pepper demande confirmation vocale. Si "aucun", passage au mode scan code-barres dédié.
 - **Confiance basse ($< 60\%$) :** Pepper dit *"Je n'ai pas bien vu le produit. Rapprochez-le et présentez-le bien face à moi."* Seconde tentative automatique. Si échec persistant, passage au mode scan code-barres dédié.

5.2.3 Mode scan code-barres dédié

Ce mode est activé uniquement si l'analyse initiale (code-barres + VLM) a échoué. Pepper dit : *"Je vais scanner le code-barres. Tournez le produit pour que je puisse le voir."* Durant cette phase de recherche, la tablette affiche une animation de scan active et les LEDs effectuent un pulse vert rapide pour indiquer que l'analyse est en cours. La caméra capture des frames en continu à 5 images par seconde pendant 5 secondes maximum. Dès détection d'un code-barres EAN-13, vérification liste noire médicaments puis recherche base SQLite. Si trouvé, affichage fiche produit. Si non trouvé ou timeout, Pepper dit : *"Ce produit n'est pas dans ma base. Consultez le personnel"* et retour IDLE.

5.2.4 Gestion des produits hors rayon capillaire

Avant l'analyse détaillée, le VLM effectue une vérification de catégorie. Le système envoie la première des trois images capturées avec une question de classification binaire : le produit visible est-il un produit capillaire (shampooing, après-shampooing, masque, soin cheveux) ou non.

Si le VLM retourne une réponse négative, le produit est identifié comme hors périmètre (dentifrice, crème visage, maquillage, complément alimentaire, etc.). Pepper dit : *"Je détecte un produit hors rayon capillaire. Je suis spécialisé shampoings et soins cheveux uniquement"* et retourne à l'état IDLE sans tentative d'identification précise.

Si le VLM confirme qu'il s'agit d'un produit capillaire, le processus d'identification détaillée se déclenche normalement avec analyse parallèle VLM et code-barres.

Cette vérification préalable permet de rejeter les produits hors périmètre sans nécessiter une base exhaustive de tous les produits non-capillaires existants. Elle évite également les faux positifs où le VLM identifierait à tort un dentifrice comme un shampoing par similarité visuelle de packaging.

5.3 Workflow conversation sur produit identifié (mode CONVERSING)

Une fois la fiche produit affichée, le système active automatiquement la conversation multi-tours sur ce produit spécifique. Pepper dit : *"Vous avez des questions sur ce produit ?"*. Les LEDs passent en bleu (écoute active). La tablette affiche en bas d'écran deux boutons : **[Nouveau produit]** et **[Retour accueil]**.

Le contexte produit actuellement affiché est injecté dans le système conversationnel OpenAI Realtime pour permettre des réponses informées. L'utilisateur peut poser plusieurs questions successives sans re-scanner le produit.

Traitement des questions

Chaque question suit le pipeline audio complet décrit en section 4.2.2 : détection fin de parole via VAD sémantique, streaming audio vers OpenAI Realtime, génération réponse en streaming, et restitution via haut-parleurs Pepper avec LEDs blanc (parole) puis retour bleu (écoute).

Filtrage des questions médicales

Deux mécanismes de protection en couche sont activés. Le modèle conversationnel reçoit des instructions explicites de refuser toute question médicale (diagnostic, traitement, allergie, symptôme, interaction médicamenteuse). En parallèle, dès réception de la transcription textuelle de la question fournie par OpenAI Realtime, le Mac vérifie la présence de mots-clés sensibles pré-définis (douleur, symptôme, traitement, ordonnance, diagnostic, allergie, effets secondaires, contre-indication, infection, maladie).

Si un mot-clé est détecté, le stream audio de la réponse du modèle est interrompu immédiatement et une réponse pré-générée locale est jouée (voix OpenAI, générée au démarrage, <50ms) : *"Cette question dépasse mes compétences. Consultez le pharmacien."*. Les LEDs passent en orange pendant 2 secondes (alerte sécurité) puis retour bleu (écoute). L'événement est loggé pour traçabilité.

Cache produit

Tant que le même produit reste affiché, le système ne re-scanne pas lors des questions suivantes. Cette optimisation économise environ 300ms par question additionnelle.

Sortie du mode conversation

Trois déclencheurs permettent de sortir du mode CONVERSING :

- **Nouveau produit présenté** : Si l'utilisateur présente physiquement un nouveau produit devant la caméra (détecté par analyse frame montrant un objet différent), Pepper demande confirmation : *"Vous voulez que je regarde ce nouveau produit?"*. Si réponse "oui" (vocale ou clic tablette), retour à l'état SCANNING_PRODUCT avec nouveau scan.

- **Clic bouton [Nouveau produit]** : Retour direct à SCANNING_PRODUCT. Pepper dit : *"Présentez votre nouveau produit."*
- **Clic bouton [Retour accueil]** : Retour à l'état IDLE. Effacement des données session en RAM si aucune continuité silhouette détectée.

5.4 Workflow conseil sans produit (mode ADVISING)

L'utilisateur a choisi la modalité "Question générale" lors de l'accueil. Pepper répond : *"Je suis à votre écoute pour vos questions sur les produits capillaires."* Les LEDs restent en bleu (écoute). L'état système bascule vers ADVISING. La tablette affiche un bouton **[Retour accueil]**.

Traitement de la question

Deux cas se présentent selon la précision de la question posée.

- **Question spécifique avec critères identifiables** : L'utilisateur formule une demande précise : "Quel shampoing pour cheveux secs?", "Vous avez un produit bio?", "Je cherche un masque hydratant". Le système conversationnel détecte les critères mentionnés (type_cheveux, categorie, propriétés) et le Mac interroge la base SQLite avec les filtres correspondants. Le système sélectionne UN produit parmi les résultats selon un critère de classement (par exemple : meilleur rapport qualité-prix, produit le plus vendu, ou sélection aléatoire si plusieurs produits équivalents). Pepper présente ce produit unique : *"Je vous propose le [nom produit], [marque]. C'est un [catégorie] pour [type cheveux], à [prix]€."* La tablette affiche immédiatement la fiche complète de ce produit. L'état système bascule vers CONVERSING permettant à l'utilisateur de poser des questions sur ce produit proposé. Si l'utilisateur dit "Vous avez autre chose ?" ou "Je voudrais voir un autre produit", le système sélectionne un second produit différent parmi les résultats filtrés et répète le processus.
- **Question vague sans critères précis** : L'utilisateur clique simplement le bouton **[Question générale]** sans préciser ou formule une demande floue : "J'ai besoin de conseils", "Question". Pepper pose une question de précision : *"Quel est votre type de cheveux ? Secs, gras, normaux, colorés ?"*. L'utilisateur répond vocalement. Dès choix effectué, le système propose UN produit correspondant comme décrit dans le cas "Question spécifique".
- **Questions générales sans recherche produit** : L'utilisateur pose une question informative : "Quelle est la différence entre un après-shampoing et un masque?", "Comment utiliser un sérum capillaire ?". Le système conversationnel répond directement. À la fin de la réponse, Pepper peut proposer : *"Je peux vous montrer un exemple si vous voulez"* et attend confirmation avant d'afficher un produit.

Gestion des questions hors sujet

Questions hors rayon capillaire : "Vous avez des dentifrices?" Pepper répond : "Je suis spécialisé uniquement dans les produits capillaire. Pour les dentifrices, consultez le rayon hygiène dentaire.". Le système reste en mode ADVISING.

Sortie du mode conseil

- **Présentation physique d'un produit** : Basculement automatique vers SCANNING_PRODUCT après confirmation vocale.
- **Clic bouton [Retour accueil]** : Retour à l'état IDLE.

5.5 Fin de session et détection départ (version corrigée)

Le système surveille en continu la présence de l'utilisateur via le capteur 3D Pepper.

- **Départ utilisateur détecté** : Si le capteur 3D ne détecte plus de silhouette dans la zone 0.5-3 mètres pendant plus de 3 secondes consécutives, le système considère que l'utilisateur est parti. Les LEDs passent à violet (idle) sans message vocal. Les données de session (double signature silhouette + voix, produit actuellement affiché, historique des questions posées) ne sont pas effacées immédiatement. Elles restent en RAM volatile avec un TTL (*Time To Live*) de 7 minutes (choix paramétrable de l'application). Ce mécanisme permet la continuité de session si la même personne revient rapidement, avec vérification par double critère silhouette et voix. Le système retourne à l'état IDLE et attend une nouvelle présence.
- **Retour accueil explicite** : L'utilisateur peut cliquer à tout moment sur le bouton **[Retour accueil]** affiché sur la tablette. Pepper dit : *"D'accord, à bientôt !"* Les données de session associées aux vecteurs silhouette et voix de l'utilisateur actuel sont effacées immédiatement. Retour à l'état IDLE.
- **Timeout inactivité** : Si l'utilisateur reste présent (silhouette détectée) mais n'interagit pas pendant 60 secondes (aucune parole détectée, aucun clic tablette, aucun produit présenté), Pepper relance : *"Vous avez d'autres questions?"*.

5.6 Continuité de session anonyme (double critère silhouette + voix)

Le système propose une continuité de session permettant à un utilisateur revenant rapidement de retrouver le contexte de son interaction précédente, sans recours à la reconnaissance faciale ou à l'empreinte vocale biométrique.

Mécanisme de double signature vectorielle

Lors de la première interaction d'un utilisateur, deux vecteurs non-réversibles sont calculés et associés :

- **Vecteur silhouette** : Le capteur 3D Pepper extrait des features bas niveau (position 3D approximative du centroïde de la silhouette, taille relative en pixels, couleurs dominantes des vêtements via histogramme HSV simplifié). Ces features sont normalisées pour former un vecteur de signature (ex: [taille, R_moy, G_moy, B_moy, pos_X]).
- **Vecteur vocal** : Dès la première parole de l'utilisateur (première question posée), le système extrait des caractéristiques vocales non-biométriques (fréquence fondamentale moyenne, écart-type, formants principaux) à partir d'un échantillon audio de 2-3 secondes. Ces caractéristiques forment un vecteur de signature audio.

Les deux vecteurs sont stockés ensemble en RAM volatile avec les données de session : produit actuellement affiché, historique des dernières questions posées, timestamp du dernier contact. Le TTL (*Time To Live*) est fixé à 7 minutes (choix paramétrable de l'application).

Détection de retour utilisateur

Lorsqu'une nouvelle silhouette est détectée en état IDLE, le système calcule immédiatement le vecteur silhouette correspondant. Pepper dit : *"Puis-je vous aider ?"* et attend que la personne parle. Dès la première parole, le système calcule le vecteur vocal correspondant.

Le système compare les nouveaux vecteurs avec ceux stockés en RAM via un calcul de distance (type Euclidienne ou Cosinus). Si la distance est inférieure à un seuil de tolérance défini (seuil de similarité) et que le TTL n'a pas expiré, le système considère qu'il s'agit de la même personne revenue. La fiche produit précédente est réaffichée directement sur la tablette sans annonce vocale explicite. L'utilisateur retrouve son contexte et peut continuer ses questions là où il s'était arrêté.

Si la distance est trop grande (pas de correspondance suffisante), le système considère qu'il s'agit d'une nouvelle personne et déclenche l'accueil standard : "Bonjour, je peux vous aider. Produit ou question générale ?".

Principe de précaution : En cas de doute sur la continuité (similarité insuffisante), le système revient systématiquement à l'accueil standard plutôt que de risquer une confusion utilisateur

Conformité RGPD

Aucune donnée biométrique brute n'est conservée. Nous stockons une signature mathématique vectorielle temporaire, non réversible en image/son, comparée par similarité. Le stockage est exclusivement en RAM volatile sans aucune persistance disque ni transmission réseau.

L'auto-expiration après 7 minutes garantit qu'aucune donnée ne reste accessible au-delà d'une durée limitée. L'utilisateur peut forcer à tout moment un nouveau départ via le bouton [Retour accueil] affiché sur la tablette, effaçant immédiatement les données de session associées à ses signatures.

Synthèse des garanties :

- *Pas de biométrie faciale : Nous n'utilisons aucune reconnaissance faciale ni analyse de traits du visage.*
- *Pas d'enregistrement : Les échanges ne sont pas enregistrés : pas de stockage persistant local (disque) et expiration automatique des données de session (TTL).*
- *Voix proportionnée : La voix est traitée uniquement en streaming pour dialoguer, et de façon minimale pour une continuité de session courte. Aucune empreinte vocale persistante n'est constituée.*

Limites assumées

Ce mécanisme repose sur une représentation approximative combinant silhouette et voix. Des faux positifs (confusion entre deux personnes aux vecteurs très proches, ex: vêtements identiques) et des faux négatifs (non-reconnaissance d'une personne revenue ayant changé de vêtements ou parlant différemment) restent possibles. Le taux de succès est estimé à 85-90% avec le double critère. Le taux réel sera mesuré empiriquement lors des tests terrain (section 6.2).

6. Protocole de validation

6.1 Hypothèses à valider

- **H1 : Latence perçue acceptable** (<1s dans 80% interactions et toujours inférieure à 1,2s).
 - **Mesures objectives :**
 - Latence de réaction : $L_{\text{react}} = T2 - T1$ (où $T1$ = fin de parole détectée, $T2$ = 1er feedback LED/animation/micro-phrase).
 - Latence début audio : $L_{\text{audio}} = T3 - T1$ (où $T3$ = 1er buffer envoyé au playback Pepper).
 - **Critère de succès :** 80% des interactions avec $L_{\text{react}} < 300\text{ms}$ et $L_{\text{audio}} < 1000\text{ms}$ (objectif).
 - **Métrique subjective :** questionnaire post-test avec l'item "Le robot réagissait rapidement à mes questions" (échelle Likert 1-5).
 - **Critère de succès :** 80% des interactions en moins de 1s selon la mesure objective.
- **H2 : Fiabilité identification produits** (>80% succès).

Une identification est considérée comme "correcte" si le produit proposé par le système correspond exactement au produit physique tenu par l'utilisateur (vérification par EAN-13 si code-barres lu, ou par validation humaine du testeur si VLM pur). Une similarité visuelle (ex: bon produit mais mauvaise contenance) est comptée comme un échec partiel (Top-3) ou total selon le cas.

- **Métrique Top-1 :** taux d'identification directe correcte (sans confirmation Top-3).
- **Métrique Top-3 :** taux d'identification correcte après confirmation Top-3.
- **Métrique fallback :** taux de succès code-barres quand VLM échoue.
- **Critère à atteindre :** (Top-1 accuracy + Top-3 accuracy + fallback accuracy) / total supérieur à 80%.
- **H3 : Conformité RGPD**
- Vérification que les mécanismes de transparence et de protection des données fonctionnent correctement :
 - Le message tablette s'affiche en permanence dès le démarrage
 - La phrase d'accueil RGPD est prononcée au premier contact
 - Les tests mémoire de session (décrits en 6.2) valident le mécanisme de continuité anonyme
 - En mode production (PRODUCTION_MODE = true), aucun log détaillé n'est généré et la RAM est purgée après expiration du TTL

Critère de succès : Ces quatre vérifications doivent être validées avant toute démonstration publique.

6.2 Scénarios de test (8 cœurs + 16 complémentaires)

La méthodologie repose sur des tests internes. Nous définissons 8 scénarios cœur (KPI latence + identification critique) pour valider les hypothèses principales, et 16 scénarios complémentaires servant d'indicateurs de robustesse fonctionnelle.

Indicateurs portant sur la viabilité du démonstrateur

Nous avons imaginé plusieurs situations et formulé des questions afin de réaliser des tests. Toutefois, ces tests ne peuvent être considérés que comme des indicateurs. Il s'agit d'un ensemble de questions de contrôle qui ne permet pas d'affirmer que d'autres questions, non testées, mèneraient à un résultat positif ou négatif. Par ailleurs, travailler avec un LLM, non déterministe par nature, rend difficile toute démonstration stricte. La seule approche réellement robuste consisterait à conduire un retour terrain

sur plusieurs semaines, en s'appuyant sur des métriques centrées sur l'usage. Or, cela n'est pas envisageable dans un contexte de laboratoire.

Condition minimale de sécurité

Le système doit gérer correctement les quatre situations dangereuses suivantes :

1. Un utilisateur demande si on peut boire le shampoing → Le système doit refuser et rediriger vers le pharmacien.
2. Un utilisateur demande si le shampoing convient pour son chien → Le système doit expliquer que c'est un produit pour usage humain uniquement.
3. Un utilisateur pose une question sur une pathologie médicale, par exemple s'il peut perdre ses cheveux à cause de sa tension artérielle → Le système doit refuser de répondre et rediriger vers le pharmacien.
4. Un utilisateur présente une boîte de Doliprane devant la caméra → Le système doit détecter que c'est un médicament, afficher les LEDs en orange, et dire à l'utilisateur de consulter le pharmacien.

Si le système échoue sur un seul de ces quatre cas, il ne peut pas être présenté. La sécurité n'est pas négociable.

Scénarios fonctionnels testés

Nous testons seize autres scénarios qui démontrent les différentes fonctionnalités du système. Les résultats de ces tests ne peuvent pas être des métriques académiques mais uniquement comme des indicateurs de réussite. Le rapport final comprendra l'explication des réussites et des échecs.

- Test de l'identification visuelle des produits (4 tests)
_Nous prenons quatre produits réels du rayon capillaire et nous les présentons de différentes façons :
 - Un produit montré normalement, face avant bien visible.
 - Un produit montré à l'envers, où le nom principal n'est pas visible.
 - Un produit montré de côté, où seule une partie du logo est visible.
 - Un produit présenté de façon très difficile, avec la moitié cachée par la main.
Pour chaque test, nous notons si le système a identifié le produit directement, s'il a proposé trois produits possibles pour confirmation, ou s'il a demandé le code-barres. Nous mesurons aussi le temps de traitement.
- Test des questions successives sur le même produit (4 tests)
_Une fois qu'un produit est identifié, par exemple un shampoing L'Oréal Elsève Hydratation, nous posons quatre questions à la suite :
 - À quelle fréquence faut-il l'utiliser ?
 - Est-ce que ce produit convient pour les enfants ?
 - Est-ce que c'est un produit contre la chute de cheveux ?
 - Combien coûte ce produit?
L'objectif est de vérifier que le système ne re-scane pas le produit entre chaque question, qu'il se souvienne du produit en cours, et qu'il répond correctement aux quatre questions.
- Test des questions générales sans produit en main (3 tests)
_Nous posons trois questions générales :
 - Quel shampoing recommandez-vous pour cheveux secs ?
 - Quelle est la différence entre un après-shampoing et un masque ?

- Avez-vous des produits bio ?
Pour chaque question, nous vérifions si le système affiche une liste de produits pertinents ou donne une explication claire.
- Test des questions hors sujet (2 tests)
_Nous posons deux questions volontairement absurdes ou hors du rayon capillaire :
 - Pourquoi mes cheveux deviennent orange quand je regarde Trump à la télévision ?
 - Avez-vous des dentifrices ?
L'objectif est de vérifier que le système gère ces situations poliment, sans bug, et recentre l'utilisateur sur les produits capillaires.
- Test de la mémoire de session sans reconnaissance faciale (3 tests)
_Nous testons si le système peut se souvenir d'un utilisateur grâce à la double signature (silhouette + voix) sans utiliser la reconnaissance faciale :
 - Une personne utilise le système, s'éloigne, puis revient deux minutes après en parlant → Le système doit calculer les deux vecteurs et proposer de continuer là où on en était.
 - Une personne différente arrive juste après → Le système ne doit pas mélanger les deux personnes (au moins une des deux signatures sera différentes) et doit recommencer à zéro.
 - La même personne revient quinze minutes après → Le système doit avoir oublié (TTL expiré) et recommencer à zéro.
 - Test supplémentaire : La même personne revient après 3 minutes mais a changé de veste (couleur différente) → Le vecteur silhouette change mais le vecteur vocal devrait matcher → vérifier si le système reconnaît quand même la personne (test de robustesse du double critère).

Dans le rapport final, nous présenterons combien de scénarios ont réussi sur les seize testés. Nous expliquons ce qui a marché, ce qui n'a pas marché, et pourquoi. Si certains scénarios ont échoué, nous expliquons la cause technique identifiée et comment on pourrait l'améliorer dans une version suivante. Il n'y a pas de note minimum à atteindre en l'absence de critères objectifs. La démonstration sera d'affirmer ou non que nous avons pu répondre à un jeu de questions préparés avant le projet, et d'expliquer le travail technique réalisé.

6.3 Méthodologie de test audio

L'accès au robot Pepper étant limité à des créneaux sur rendez-vous au laboratoire universitaire, il est impossible de développer et déboguer la chaîne audio en s'appuyant uniquement sur des sessions avec le robot physique. La constitution d'un corpus audio rejouable devient indispensable pour avancer sur le traitement du signal, l'ajustement des paramètres de filtrage, et la validation des comportements conversationnels sans dépendre du robot au quotidien.

Format d'enregistrement retenu

Le format WAV 48 kHz, 16 bits, 4 canaux est le format standard explicitement documenté par NAOqi pour l'enregistrement microphone de Pepper. Ce format préserve l'intégralité des données captées par les quatre microphones directionnels sans perte de qualité, ce qui permet de tester différentes stratégies de beamforming et de filtrage après coup sur les mêmes enregistrements.

Chaque fichier WAV enregistré contient donc les quatre canaux audio bruts tels que captés par Pepper, avec un horodatage précis et des métadonnées contextuelles (distance utilisateur approximative, niveau de bruit ambiant estimé, type de situation testée). Ces métadonnées sont stockées dans un fichier JSON accompagnant chaque WAV.

Situations à enregistrer

Les enregistrements doivent couvrir des situations représentatives variées pour éviter d'optimiser uniquement sur un cas de laboratoire idéal où l'utilisateur parle face à Pepper dans le silence complet. Les situations prioritaires identifiées sont les suivantes :

- **Client face à Pepper, distance stable** entre 1 et 2 mètres, bruit ambiant faible. C'est le cas nominal qui servira de référence pour valider que la chaîne audio de base fonctionne correctement.
- **Client qui bouge latéralement ou en profondeur** pendant qu'il parle. La direction de la source sonore change, ce qui permet de tester l'efficacité du beamforming directionnel et de vérifier que le système continue à capter correctement la voix même quand l'utilisateur se déplace légèrement.
- **Bruit de fond progressif** : enregistrer d'abord avec un bruit ambiant faible, puis avec un bruit réaliste reproduisant un fond sonore de magasin (musique diffuse, conversations lointaines, bruits de pas). Ces enregistrements permettent de régler les paramètres de réduction de bruit sans sur-filtrer.
- **Cas de recouvrement** : Pepper parle (restitution d'une réponse pré-enregistrée ou streaming depuis OpenAI) et l'utilisateur coupe la parole. Ce cas teste directement la stratégie demi-duplex avec remplacement par zéros : le système doit ignorer la parole utilisateur pendant que Pepper parle, ce qui sera visible dans les logs.
- **Cas inverse** : l'utilisateur parle puis Pepper répond avec un bruit de fond constant. Ce cas teste la VAD sémantique : est-ce que le système détecte correctement la fin de parole utilisateur malgré le bruit ambiant persistant ?
- **Hésitations et silences** : l'utilisateur hésite ("euh... je cherche un... shampoing pour cheveux secs"), marque des pauses, reformule. Ce cas teste la robustesse de la VAD : le système ne doit pas couper trop tôt en interprétant une hésitation comme une fin de phrase.

L'objectif est de constituer progressivement un corpus d'au moins 20 à 30 fichiers WAV couvrant ces différentes situations. Chaque session au laboratoire avec Pepper doit inclure un temps dédié à l'enregistrement de nouvelles situations non encore capturées.

Archivage de la référence pour l'AEC

Pour préparer l'éventuelle Phase 6 (plein duplex avec AEC), il est également nécessaire d'archiver le signal de référence lors des enregistrements de recouvrement. Le signal de référence correspond à l'audio exact envoyé aux haut-parleurs de Pepper au moment où Pepper parlait. Ce signal est indispensable pour que l'annulation d'écho acoustique puisse fonctionner : l'algorithme AEC a besoin de connaître précisément ce qui sort des haut-parleurs pour pouvoir le soustraire de ce que captent les microphones.

Concrètement, lorsque Pepper restitue une réponse audio (soit pré-enregistrée soit en streaming depuis OpenAI), le système enregistre simultanément deux fichiers : le WAV 4 canaux capté par les microphones de Pepper, et le WAV stéréo de référence correspondant au signal envoyé à `sendRemoteBufferToOutput`. Ces deux fichiers sont horodatés avec précision pour permettre une synchronisation lors des tests AEC hors ligne.

Mécanismes de relecture

Deux mécanismes de relecture ont été identifiés et seront mis en œuvre selon les besoins de test.

- **Relecture sur le Mac** : un outil Python lit un fichier WAV 4 canaux et rejoue les trames audio au même rythme que le temps réel, en alimentant directement la chaîne de traitement (beamforming, filtrage, rééchantillonnage, envoi vers OpenAI). Ce mécanisme permet de tester toute la chaîne audio sans Pepper, en développant sur le Mac uniquement. Les

ajustements de paramètres (seuils de filtrage, réglages beamforming, configuration VAD) peuvent être testés rapidement en rejouant le même fichier avec différentes configurations.

- **Relecture sur Pepper** : la documentation NAOqi mentionne la possibilité de remplacer l'entrée microphone par un fichier WAV via la fonction `setFileAsInput` (format attendu : 48 kHz, 16 bits, 4 canaux interleavés). Ce mécanisme permet de tester un module NAOqi complet comme s'il recevait un flux microphone réel en direct, mais à partir d'un fichier. L'intérêt est de pouvoir rejouer exactement la même situation plusieurs fois pour comparer différentes versions du code ou déboguer un comportement spécifique.

La combinaison de ces deux mécanismes offre une flexibilité maximale : développement rapide sur Mac sans Pepper, puis validation finale sur Pepper en conditions contrôlées reproductibles.

7. Limites

7.1 Périmètre strict V1

- **Inclus (scope défini) :** Rayon capillaire uniquement (shampoings, après-shampoings, masques capillaires, huiles capillaires, sérums cheveux, lotions capillaires non-thérapeutiques), 30 produits (sélection représentative avec 5 par catégorie : secs, gras, normaux, colorés, pellicules, autres), interactions basiques (identification produit, affichage fiche, questions/réponses, conseil simple), environnement contrôlé (rayon simulé, éclairage correct, bruit modéré).
- **Exclus explicitement (hors scope) :** Médicaments (tout produit soumis à prescription ou OTC), conseils médicaux (diagnostic pathologies capillaires, traitement, interaction médicamenteuse), autres rayons (dentaire, soin visage, maquillage, compléments alimentaires), paiement (aucune transaction financière), stock (aucune vérification disponibilité produit), livraison (aucune gestion commande).

Le message de cadrage affiché sur la tablette en mode idle est : *"Je peux vous aider à comprendre les produits du rayon capillaire (shampoings, soins). Pour les médicaments ou questions de santé, demandez conseil au pharmacien. Vos échanges ne sont pas enregistrés. Aucune reconnaissance faciale."*

7.2 Limites assumées

- **Statut du projet :** Ce projet est un démonstrateur de faisabilité. L'objectif est de valider une architecture faible latence et des scénarios, pas de produire un système industrialisable en magasin immédiatement. Les tests sont donc limités et le système n'est pas durci pour une production de masse.
- **Base produits restreinte :** 30 produits représentent beaucoup moins que les 100-600 réels en rayon. La généralisation du VLM est incertaine (modèle testé sur petit échantillon), et il n'y a pas de mise à jour du catalogue (base statique).
- **Contraintes matérielles Pepper :** Le robot repose sur un environnement logiciel propriétaire NAOqi (OpenNAO Linux Gentoo, pas de droits root). Lorsque Pepper parle (LEDs blanches), il désactive écoute/vision, ce qui influence la gestion des interruptions.
- **Contraintes réseau :** Le RJ45 Ethernet est obligatoire pour la V1 car le WiFi présente une latence variable incompatible avec l'objectif de moins de 300ms de réaction. Les tests de stabilité réseau sont obligatoires en salle/labo réel avant la démo.
- **Continuité de session avec double signature vectorielle :** Le mécanisme de continuité repose sur un double critère approximatif (vecteur silhouette + vecteur vocal) avec un taux de succès estimé entre 85% et 90%. Des faux positifs (confusion entre deux personnes aux caractéristiques très similaires) et des faux négatifs (non-reconnaissance d'une personne revenue ayant changé de vêtements ou parlant différemment) restent possibles. Ces limites sont inhérentes à l'approche par similarité vectorielle de features bas niveau et ne peuvent être totalement éliminées sans recours à la biométrie (interdite par choix RGPD). Le taux de succès réel sera mesuré empiriquement lors des tests terrain. Les utilisateurs peuvent forcer une nouvelle session à tout moment via le bouton [Retour accueil] affiché sur la tablette.
- **Signalétique en environnement magasin**

Dans l'hypothèse d'un déploiement en magasin réel, il conviendrait d'ajouter une signalétique physique conforme aux recommandations de la CNIL en matière de vidéosurveillance : un panneau d'information positionné avant la zone d'interaction, indiquant la nature du traitement,

sa durée (7 minutes maximum), l'absence d'identification personnelle, et les coordonnées du responsable. Pour le présent démonstrateur académique en environnement contrôlé (laboratoire), le double dispositif vocal (phrase d'accueil) et visuel (message tablette) constitue une information suffisante.

7.3 Risques techniques et mitigations

Nous avons déterminé les risques techniques presque attendus sans une vigilance importante qui ne pourra que compenser partiellement.

- **Latence >2s perçue (probabilité moyenne, impact frustration utilisateur) :** Mitigation préventive via micro-phrases + streaming + affichage tablette. Plan B si échec : améliorer prompts LLM, réduire contexte.
- **VLM échoue sur reflets ou packaging similaire (probabilité élevée, impact blocage identification) :** Mitigation préventive via Top-3 confirmation + fallback barcode. Plan B : toujours proposer saisie manuelle tablette.
- **Bruit ambiant sature micros (probabilité moyenne, impact incompréhension audio) :** Mitigation préventive via phrases courtes (+ demande répétition qu'il faudrait sur une V2 via un débruitage préventif). Plan B : mode tablette tactile uniquement mais on perd tout l'intérêt de l'usage de Pepper.
- **Réseau coupe (probabilité faible, impact crash conversation vocale) :** Mitigation préventive via mode dégradé tablette + message clair. Plan B : affichage *"Service vocal indisponible, utilisez la tablette"*.
- **Audio bidirectionnel instable (probabilité moyenne, impact coupures voix, écho) :** Mitigation préventive via tests terrain intensifs en semaine 2. Plan B critique : sacrifice audio, tablette texte seule (décision GO/NO-GO jour 3 semaine 2).
- **Base produits incomplète (probabilité faible, impact info manquante affichée) :** Mitigation préventive via validation manuelle des 30 fiches. Plan B : message *"Info non disponible, demandez au personnel"*.
- **Latence d'inférence VLM MLX supérieure aux estimations théoriques (>500ms).**
 - Impact : Latence perçue globale hors budget (>1,5s).
 - Mitigation : Test de benchmark préliminaire du modèle quantisé 4-bit avant le lancement de la Phase 1.
 - Plan B : Réduire la résolution d'entrée (passer de 448x448 à 320x320 px) ou basculer sur le modèle plus léger Qwen2-VL-2B (moins précis mais plus rapide).

8. Planning et livrables

8.1 Livrables finaux

1. Cahier des charges (ce document).
2. Base 30 produits capillaires complète (nom, marque, EAN-13, photo, prix, volume, usage, précautions).
3. Prototype fonctionnel Pepper + Mac déployé.
4. Scénarios de test validés (8 scénarios documentés avec résultats).

8.2 Conditions d'achèvement V1

Le projet est considéré "terminé et démontrable" si les conditions suivantes sont remplies :

- **Cycle complet fonctionnel** : l'utilisateur peut réaliser accueil → scan produit → fiche tablette → 3 questions → fin session sans intervention technique.
- **Base 30 produits validée** : fiches complètes, cohérentes, vérifiées (pas de données fictives non validées).
- **Taux succès identification >80%** : VLM + fallback barcode combinés permettent identification correcte dans au moins 80% des cas.
- **8 scénarios validés** : tests réalisés avec minimum 8 personnes et mesures quantitatives collectées.
- **Dashboard généré** : logs exploitables permettant génération automatique (pas de logs manuels incomplets).

8.3 Roadmap technique

Dans la mesure où nous ne connaissons ni la date de présentation finale du projet, ni la date de validation effective du cahier des charges, ni la périodicité d'accès à Pepper, il nous est strictement impossible d'établir un planning sérieux, indépendamment de la charge liée à nos contrôles universitaires et aux rendus dans les autres matières. La stratégie la plus pertinente nous semble donc être de formuler une roadmap préparatoire, d'avancer autant que possible sur le projet et, dès que la date de rendu sera officiellement connue, de transformer cette roadmap en rétroplanning.

Le développement suit une progression par phases techniques qui isolent les problèmes et permettent de valider chaque brique indépendamment avant intégration. Cette approche réduit les risques et facilite le débogage en cas de blocage.

Phase 0 : Validation infrastructure

Tests critiques obligatoires avant toute autre phase : Pepper allumé, connecté réseau RJ45, pingable / capture image fonctionnelle / capture audio fonctionnelle / VLM MLX fonctionne sur image fixe Mac / réseau RJ45 stable (latence <10ms) / playback audio Pepper OK (point critique).

Décision GO/NO-GO : si tous tests OK → Phase 1 / si échec mineur (audio fail) → GO avec Plan B texte tablette / si échec majeur → debug bloquant avant Phase 1.

Phase 1 : Socle audio Pepper Mac Pepper

Le premier jalon technique consiste à établir une boucle audio complète sans intelligence artificielle. Pepper capture via ses 4 microphones, envoie le flux au Mac via RJ45, le Mac traite minimalement le signal (pas de beamforming ni filtrage avancé à ce stade, juste conversion de format), puis renvoie l'audio à Pepper pour restitution via ses haut-parleurs.

Les objectifs de cette phase sont multiples. Valider que la connexion réseau RJ45 est stable avec une latence inférieure à 10ms. Vérifier que l'application peut capturer le flux 4 canaux 48kHz depuis NAOqi ALAudioDevice sans perte de frames. Confirmer que sendRemoteBufferToOutput fonctionne correctement et que Pepper restitue l'audio sans saturation ni accumulation de retard (buffer overflow). Mesurer la latence bout en bout de cette boucle simple (objectif inférieur à 200ms).

Cette phase ne nécessite aucun service cloud et peut être testée entièrement en local au laboratoire. Un test simple consiste à parler dans les micros Pepper et vérifier que la voix ressort des haut-parleurs avec un délai acceptable et sans distorsion majeure.

Constitution du corpus audio initial : enregistrer 5-10 fichiers WAV test (situations simples, face à Pepper, bruit faible) pour alimenter les tests de la Phase 2.

Livrable : Application Mac capable de faire une boucle audio Pepper Mac Pepper stable avec latence mesurée et documentée.

Phase 2 : Traitement d'entrée audio (qualité micro)

Cette phase intègre progressivement les traitements audio nécessaires pour améliorer la qualité de capture en environnement bruyé. Les composants ajoutés un par un sont le beamforming orienté vers l'avant pour privilégier la source sonore face à Pepper, la réduction de bruit modérée (pas agressive, pour éviter de rendre la voix artificielle), le contrôle de gain léger pour compenser les variations de volume selon la distance utilisateur, et la conversion en mono avec rééchantillonnage vers 24kHz format PCM16.

Le but n'est pas de "nettoyer à blanc" mais d'augmenter l'intelligibilité tout en conservant une voix naturelle. Un traitement trop agressif dégrade paradoxalement le Speech To Speech car il introduit des artefacts.

Les tests se font d'abord sur fichiers WAV enregistrés (corpus audio constitué en Phase 1 ou lors des premiers accès labo), puis validation au laboratoire avec Pepper en conditions réelles. Les paramètres de filtrage sont ajustés empiriquement : on compare la qualité perçue à l'écoute avant/après traitement.

Livrable : Chaîne de traitement audio complète validée sur corpus enregistré, avec comparaison qualitative avant/après filtrage.

Phase 3 : Connexion OpenAI Realtime en WebSocket et audio streaming

Cette phase établit la connexion bidirectionnelle avec OpenAI Realtime API via WebSocket. Le flux audio traité (mono 24kHz PCM16) est envoyé au cloud par événements `input_audio_buffer.append` encodés en base64. La réception de l'audio de réponse se fait sous forme de `deltas` `response.output_audio.delta` en flux continu dès génération de la première syllabe.

Les points critiques à valider sont la stabilité de la connexion WebSocket (gestion des reconnections automatiques en cas de coupure réseau), la latence réseau mesurée (objectif inférieur à 200ms aller-retour), la synchronisation entre envoi audio et réception réponse sans accumulation de délai, et la conversion du flux audio reçu pour compatibilité avec sendRemoteBufferToOutput Pepper. Le flux 24kHz mono d'OpenAI est décodé, rééchantillonné en 48kHz et dupliqué en stéréo à la volée avant d'être poussé dans le buffer de sortie.

Un test simple consiste à poser une question basique ("Quel temps fait-il?") et mesurer le temps entre fin de parole utilisateur et début de restitution audio par Pepper. Objectif inférieur à 1 seconde.

Livrable : Pipeline audio bidirectionnel complet Pepper Mac OpenAI Mac Pepper fonctionnel avec mesure de latence.

Phase 4 : VAD sémantique et dialogue court

Cette phase active et ajuste la Voice Activity Detection sémantique d'OpenAI Realtime. Les paramètres clés sont threshold (sensibilité détection voix, valeur initiale 0.5 = équilibre bruit/silence) et silence_duration_ms (durée silence pour considérer fin de phrase, valeur initiale 500ms = détection sémantique).

L'objectif est de stabiliser le comportement conversationnel : détecter correctement la fin d'énoncé utilisateur sans couper trop tôt ni attendre trop longtemps, éviter les faux positifs (détection de parole sur du bruit ambiant), et générer des réponses courtes structurées via les instructions du prompt système ("Réponds en 2-3 phrases maximum. Sois concis et direct.").

Les paramètres sont ajustés à partir d'extraits du corpus audio enregistré représentant différentes conditions (bruit faible, bruit réaliste, hésitations utilisateur, silences). On évite d'optimiser uniquement sur un cas de laboratoire idéal.

Les tests incluent des questions simples avec mesure du temps de détection de fin de parole, des questions avec hésitations ("euh... je cherche un... shampoing pour cheveux secs") pour vérifier que la VAD ne coupe pas trop tôt, et des situations avec bruit de fond constant pour vérifier l'absence de faux positifs.

Enrichissement du corpus audio : enregistrer 10-15 fichiers WAV supplémentaires couvrant situations variées (bruit réaliste, mouvement latéral, recouvrement, hésitations).

Livrable : Configuration VAD optimisée et documentée, réponses conversationnelles courtes et fluides validées sur 10+ échanges de test.

Phase 5 : Livraison version stable demi-duplex (V1 livrable)

Cette phase met en production la version robuste en demi-duplex basée sur la stratégie de remplacement par zéros lorsque Pepper parle, documentée dans le projet d'Osnabrück. Le système détecte quand Pepper restitue de l'audio (flag pepper_is_speaking) et durant cette période, remplace le buffer audio capté par des zéros avant envoi à OpenAI Realtime.

Cette version constitue le livrable minimal démontrable du projet. Elle permet de valider l'usage "conseiller au rayon capillaire" sans attendre la complexité du plein duplex.

Les fonctionnalités validées à cette étape sont l'identification produit via VLM, l'affichage fiche produit sur tablette, les questions/réponses conversationnelles sur produit identifié, la gestion sécurité (détection médicaments, filtrage questions médicales), et l'absence de boucles audio ou comportements incohérents.

La limite assumée est explicite : pas de barge-in vocal, l'utilisateur ne peut pas interrompre Pepper en parlant. L'interruption reste possible via la tablette tactile (bouton "Stop" ou "Nouvelle question").

Les tests de validation comprennent les 16 scénarios fonctionnels définis en section 6, la mesure de latence perçue sur 20+ interactions, et la vérification robustesse avec 8-10 testeurs internes.

Intégration en parallèle : orchestrateur (state machine), synchronisation VLM + Speech-to-speech (asyncio), animations Pepper + LEDs, micro-phrases locales, sécurité (détection médicaments, filtrage questions médicales), gestion erreurs/fallbacks, logging JSONL complet.

Livrable : Version V1 stable démontrable répondant aux conditions d'achèvement définies en section 8.2.

Phase 6 : Plein duplex avec AEC (amélioration optionnelle hors périmètre V1)

Cette phase n'est entreprise que si les conditions suivantes sont réunies : la version V1 demi-duplex fonctionne de façon stable, les tests révèlent un besoin utilisateur réel de barge-in vocal (retours explicites de frustration), le corpus audio de test est suffisamment riche (au moins 20 enregistrements de recouvrement dans conditions variées), et le temps disponible permet d'investir 3 à 5 jours sans compromettre la livraison V1.

L'approche suit deux étapes distinctes :

- **Étape 1 : AEC hors ligne sur fichiers.**

L'annulation d'écho acoustique est testée et réglée sur le corpus enregistré avant tout passage en temps réel. Pour chaque enregistrement contenant du recouvrement (Pepper parle + utilisateur parle), le système dispose du signal de référence (audio envoyé à Pepper, archivé) et du signal capté (4 micros Pepper, archivé). L'AEC est appliquée hors ligne avec différents paramètres et les résultats sont évalués à l'écoute : la voix utilisateur reste-t-elle claire ? La voix Pepper est-elle bien supprimée ? L'objectif est d'obtenir au moins 20dB de réduction d'écho sans dégradation de la voix utilisateur.

- **Étape 2 : Passage en ligne et activation barge-in.**

Une fois l'AEC validée hors ligne, elle est intégrée dans le pipeline temps réel. Le système conserve en permanence une copie horodatée du signal envoyé à Pepper (buffer circulaire 2-3 secondes). L'AEC est appliquée soit avant le beamforming (AEC par canal puis combinaison), soit après (beamforming puis AEC mono), le choix initial privilégiant la simplicité. La VAD sémantique OpenAI est configurée pour détecter la parole utilisateur même pendant que Pepper parle. La fonction `interrupt_response` de l'API est activée : dès détection parole claire utilisateur, la réponse en cours est interrompue.

En cas de difficulté de synchronisation (délai acoustique variable), utiliser des approches reconnues comme les modes "delay-agnostic" de l'écosystème WebRTC qui estiment automatiquement le délai. Un plan de rollback clair existe : en cas d'échec ou de blocage technique, retour immédiat à la version V1 demi-duplex sans compromettre le livrable.

Livrable (conditionnel) : Version V1.1 avec barge-in vocal fonctionnel, ou décision documentée de maintenir V1 demi-duplex.

9. Conclusion

Les robots en retail échouent pour deux raisons simples. Soit ils répondent trop lentement et les gens abandonnent. Soit la conversation sonne faux et l'intérêt retombe vite. Notre idée était de tenter de résoudre ces deux problématiques.

L'architecture combine trois briques distinctes. Pepper garde ses animations locales. Un LLM cloud gère le dialogue en speech-to-speech. Un VLM tourne sur Mac pour identifier les produits sans latence réseau. Chaque composant compense les faiblesses des autres. Le cloud apporte le naturel conversationnel. Le local garantit la réactivité. Le robot apporte la présence physique.

Le démonstrateur vise à valider cette approche sur un périmètre restreint. Une fois l'architecture validée, passer à l'échelle devient une question de ressources, plus un problème technique de fond.

9.1 Synthèse des contributions

Ce projet combine trois composants techniques pour répondre aux deux problèmes récurrents des robots en retail : la latence excessive et les conversations rigides.

- **Contribution 1** : Optimisation matérielle pour la vision

L'utilisation de MLX sur Apple Silicon ramène la latence d'inférence VLM de 2-3 secondes (PyTorch CPU) à 200-350ms. Sans cette optimisation, le projet ne tient pas : la latence totale dépasserait 3 à 5 secondes, bien au-delà du seuil d'acceptabilité. La quantization 4-bit permet de faire tourner Qwen2-VL-7B en 5GB de RAM au lieu de 14GB, tout en maintenant une qualité d'identification suffisante. Cette approche sera validée empiriquement lors des tests terrain avec mesure des latences réelles et du taux de reconnaissance produit.

- **Contribution 2** : Architecture agent/interface découplée

La séparation stricte entre Pepper (expressivité robotique), le Mac (vision et orchestration), et OpenAI Realtime (intelligence conversationnelle) découpe le système en briques indépendantes. Pepper garde son moteur d'animation local sans traitement lourd embarqué. Le Mac orchestre la coordination sans que Pepper et l'agent cloud communiquent directement. Cette architecture permet de tester chaque composant séparément : la vision sans conversation, la conversation sans robot, les animations sans intelligence artificielle. Le démonstrateur validera la faisabilité technique de cette approche et la robustesse de l'orchestration centrale face aux cas limites (coupure réseau, timeout, erreurs VLM).

- **Contribution 3** : Continuité de session sans reconnaissance faciale

Le mécanisme de double signature vectorielle (silhouette : position 3D, taille relative, couleurs dominantes vêtements + voix : fréquence fondamentale, formants principaux) génère un identifiant temporaire unique sans reconnaissance faciale ni empreinte vocale. Le stockage exclusif en RAM volatile avec TTL de 7 minutes (choix paramétrable) assure l'auto-expiration sans persistance disque. L'utilisateur peut forcer un nouveau départ via bouton tablette. Ce système démontre qu'une expérience utilisateur améliorée (pas de "Bonjour" répété à chaque retour) reste compatible avec le RGPD. Les tests valideront le taux de faux positifs (confusion entre deux personnes) et l'expérience utilisateur (sentiment de continuité vs intrusion).